รายงานโครงงาน

PROJECT REPORT

ระบบตรวจจับและครอปป้ายทะเบียนรถไทย ด้วยโมเดล YOLO11 และ OpenCV

Thai License Plate Detection and Cropping System using YOLO11 Object Detection and OpenCV



นำเสนอโดย

นายธิติวุฒิ สุกทอง

รหัสนักศึกษา 6724650293

พฤศจิกายน 2569

สารบัญ

บทที่ 1 ชื่อโครงงานและบทนำ 3

1.1 ชื่อโครงงาน 3

1.2 ที่มาและความสำคัญ 3

1.3 วัตถุประสงค์ 3

1.4 Input และ Output ของระบบ 4

1.5 ขอบเขตของโครงงาน 4

1.6 เครื่องมือและเทคโนโลยีที่ใช้ 4

บทที่ 2 Dataset: ได้มาจากไหนและเตรียมอย่างไร 6

2.1 ภาพรวมของชุดข้อมูล 6

2.2 ทำไมเลือก Wikimedia Commons 6

2.3 ขั้นตอนการเก็บข้อมูล 5 รอบ 7

2.4 เกณฑ์การคัดเลือกรูป (Passed / Rejected) 7

2.5 สัญญาอนุญาตและการให้เครดิต 8

2.6 บทเรียนสำคัญจากการเก็บข้อมูล 8

2.7 ตัวอย่างข้อมูลในชุด Dataset 9

2.8 การตีกรอบ (Labeling) 9

2.9 การแบ่งชุดข้อมูล (Data Split) 10

2.10 dataset_manifest.csv: บันทึกที่มาของทุกรูป 11

บทที่ 3 Algorithm: YOLO11 และเทคนิคที่เกี่ยวข้อง 12

3.1 Object Detection คืออะไร 12

3.2 วิวัฒนาการของ YOLO และแนวคิด One-Stage Detector 12

3.3 หลักการทำงานของ YOLO 13

3.4 สถาปัตยกรรม YOLO11 14

3.5 Loss Function: โมเดลเรียนรู้จากอะไร 16

3.6 IoU (Intersection over Union) 17

3.7 NMS (Non-Maximum Suppression) 17

3.8 Preprocessing: Letterbox Resize 18

3.9 Transfer Learning 18

3.10 Data Augmentation 19

3.11 ตัวชี้วัดการประเมินผล (Evaluation Metrics) 19

3.12 K-Fold Cross-Validation 20

3.13 การครอปภาพด้วย OpenCV 20

3.14 ข้อดีและข้อเสียของ YOLO11 21

3.15 เปรียบเทียบกับวิธีอื่นในการหาป้ายทะเบียน 21

บทที่ 4 Code: ตัวอย่างโค้ดสำคัญของโครงงาน 10 ส่วน 23

4.1 กันรูปซ้ำด้วย SHA-256 (ingest_images.py) 23

4.2 ย่อรูปและบันทึกเป็นมาตรฐานเดียวกัน (ingest_images.py) 23

4.3 จัดกลุ่มรถคันเดียวกันจากชื่อไฟล์ต้นฉบับ (split_folds.py) 24

4.4 แจกกลุ่มรถเข้า 5 Fold อย่างสมดุล (split_folds.py) 24

4.5 สร้างไฟล์ data.yaml ให้แต่ละ fold (split_folds.py) 25

4.6 ฝึกและประเมินโมเดลแต่ละ Fold (train_yolo11.py) 25

4.7 ตรวจจับป้ายและเลือกกรอบหลัก (predict_and_crop.py) 26

4.8 ครอปป้ายด้วย OpenCV (crop_utils.py) 26

4.9 คำนวณ IoU เพื่อประเมินอัตราครอปสำเร็จ (evaluate_test_set.py) 27

4.10 แปลงกรอบที่ผู้ใช้วาดบนเว็บเป็น YOLO format (feedback_utils.py) 28

บทที่ 5 แผนภาพ (Pipeline) ของทั้งโครงงาน 29

5.1 ภาพรวมของทั้งโครงงาน 29

5.2 Inference Pipeline: จากรูปรถถึงป้ายที่ครอปแล้ว 30

5.3 วงจร Human-in-the-Loop 31

5.4 โครงสร้างโฟลเดอร์ของโปรเจกต์ 32

บทที่ 6 บรรยายงาน: ขั้นตอนการดำเนินงานทั้งหมด 33

6.1 Data Preparation (การเตรียมข้อมูล) 33

6.2 Data Cleaning (การทำความสะอาดข้อมูล) 33

6.3 Data Transformation (การแปลงข้อมูล) 34

6.4 Train Model (การฝึกโมเดล) 35

6.5 Trained Model (ผลของโมเดลที่ฝึกแล้ว) 36

6.6 Save Model (การบันทึกและเลือกโมเดล) 37

6.7 Evaluation (ประเมินผลกับชุดทดสอบ) 38

6.8 Deployment: การนำโมเดลไปใช้งาน 40

6.9 Feedback Loop และการพัฒนาโมเดลรอบถัดไป 42

6.10 ข้อจำกัดและแนวทางพัฒนาต่อ 43

6.11 สรุป 43

บรรณานุกรม 44

(หากหมายเลขหน้าไม่ขึ้น: คลิกขวาที่สารบัญ ⟶ Update Field ⟶ Update entire table)

บทที่ 1 ชื่อโครงงานและบทนำ

1.1 ชื่อโครงงาน

ภาษาไทย: ระบบตรวจจับและครอปป้ายทะเบียนรถไทยด้วยโมเดล YOLO11 และ OpenCV

ภาษาอังกฤษ: Thai License Plate Detection and Cropping System using YOLO11 and OpenCV

ผู้จัดทำ: นายธิติวุฒิ สุกทอง รหัสนักศึกษา 6724650293

1.2 ที่มาและความสำคัญ

ป้ายทะเบียนรถเป็นข้อมูลสำคัญที่ใช้ระบุตัวตนของรถยนต์ในงานหลายประเภท เช่น
ระบบควบคุมการเข้าออกอาคารและที่จอดรถ ระบบเก็บค่าผ่านทางอัตโนมัติ
การตรวจจับการกระทำผิดกฎจราจร และการสืบสวนเหตุอาชญากรรม ขั้นตอนแรกของระบบเหล่านี้คือ **การหาตำแหน่งป้ายทะเบียนในภาพ** ก่อนที่จะนำป้ายไปอ่านตัวอักษร (OCR) หรือตรวจสอบกับฐานข้อมูล หากตรวจจับตำแหน่งป้ายผิดพลาด ขั้นตอนถัดไปทั้งหมดก็จะผิดตามไปด้วย

วิธีดั้งเดิมที่ใช้ การประมวลผลภาพแบบกำหนดกฎเอง (เช่น หาขอบสี่เหลี่ยม หรือกรองสีพื้นป้าย) มักทำงานได้ไม่เสถียรเมื่อแสง มุมกล้อง หรือสีของรถเปลี่ยนไป ปัจจุบันเทคนิค Deep Learning ด้าน Object Detection โดยเฉพาะตระกูล YOLO (You Only Look Once)
สามารถเรียนรู้ลักษณะของป้ายทะเบียนจากตัวอย่างจริง และให้ผลที่แม่นยำพร้อมทำงานได้รวดเร็ว
โครงงานนี้จึงนำโมเดล YOLO11 รุ่นเล็กที่สุด (yolo11n) มาฝึกให้ตรวจจับป้ายทะเบียนรถไทย แล้วใช้ OpenCV ตัดเฉพาะส่วนป้ายออกมาเป็นรูปใหม่ขนาดมาตรฐาน พร้อมนำไปใช้งานต่อ

1.3 วัตถุประสงค์

- พัฒนาระบบที่รับรูปรถ 1 คัน แล้วหาตำแหน่งป้ายทะเบียนไทยในรูปได้อัตโนมัติ
- ตัดภาพเฉพาะป้ายทะเบียนออกมาเป็นรูปใหม่ขนาด 400×200 พิกเซล
- ฝึกและประเมินโมเดลอย่างเป็นระบบด้วย 5-Fold Cross-Validation และชุดทดสอบแยกต่างหาก เพื่อให้ผลเชื่อถือได้
- จัดทำชุดข้อมูลจากแหล่งที่เปิดสิทธิ์การใช้งาน พร้อมบันทึกที่มาและสัญญาอนุญาตของทุกรูป
- พัฒนาส่วนติดต่อผู้ใช้ทั้งแบบ Command Line และ Web Application (Streamlit) พร้อมกลไกรับ feedback จากผู้ใช้เพื่อพัฒนาโมเดลรอบถัดไป

1.4 Input และ Output ของระบบ

ตารางที่ 1 ข้อมูลนำเข้าและผลลัพธ์ของระบบ

รายการ	รายละเอียด
Input	รูปภาพรถ 1 คัน (JPG / PNG) ที่ถ่ายจากด้านหน้าตรงหรือด้านหลังตรง และเห็นป้ายทะเบียนชัดเจน
Output (เจอป้าย)	รูปป้ายทะเบียนที่ครอปแล้ว ขนาด 400×200 พิกเซล บันทึกเป็นไฟล์ <ชื่อรูป>_plate.jpg พร้อมค่าความมั่นใจ (confidence)
Output (ไม่เจอป้าย)	ข้อความ "ไม่พบป้ายทะเบียน" และให้ผู้ใช้ถ่ายใหม่ หรือวาดกรอบป้ายเองบนเว็บเพื่อส่งเป็นข้อมูลใหม่

1.5 ขอบเขตของโครงงาน

- ตรวจจับ **คลาสเดียว** คือ license_plate (ป้ายทะเบียน) ไม่รวมการอ่านตัวอักษรบนป้าย (OCR)
- รองรับรูปรถ 1 คัน ถ่ายหน้าตรงหรือหลังตรง ป้ายอยู่ในเฟรมครบ หากพบหลายป้ายจะเลือกป้ายที่ใหญ่ที่สุด (รถคันหลัก)
- ชุดข้อมูลเก็บจากอินเทอร์เน็ต (Wikimedia Commons) ตามที่อาจารย์อนุญาต จำนวน 250 รูปที่ผ่านเกณฑ์
- ไม่รองรับมุมถ่ายเฉียงมาก และป้ายสีแดงมีตัวอย่างน้อย โมเดลอาจพลาดได้ (วิเคราะห์ไว้ในบทที่ 6)

1.6 เครื่องมือและเทคโนโลยีที่ใช้

ตารางที่ 2 เครื่องมือและไลบรารีหลักของโครงงาน

เครื่องมือ	เวอร์ชัน / รายละเอียด	หน้าที่ในโครงงาน
Python	3.13	ภาษาหลักของสคริปต์ทั้งหมด
Ultralytics	8.4.190	ไลบรารี YOLO11 สำหรับฝึก ประเมิน และตรวจจับ
OpenCV	opencv-python 5.0	อ่านรูป ครอป และปรับขนาดภาพ
Pillow	12.0	ย่อรูป (resize) และบันทึก JPEG ตอนนำเข้าข้อมูล
Streamlit	1.51.0	เว็บแอปอัปโหลดรูป แสดงผล และรับ feedback

เครื่องมือ	เวอร์ชัน / รายละเอียด	หน้าที่ในโครงงาน
makesense.ai	เว็บแอป (ไม่ต้องติดตั้ง)	ตีกรอบป้ายทะเบียน (labeling) และส่งออกเป็น YOLO format
Apple MPS	GPU ของ Mac	เร่งความเร็วการฝึกโมเดล (device = mps)

บทที่ 2 Dataset: ได้มาจากไหนและเตรียมอย่างไร

2.1 ภาพรวมของชุดข้อมูล

ชุดข้อมูลของโครงงานนี้คือ **รูปถ่ายรถยนต์ที่เห็นป้ายทะเบียนไทย จำนวน 250 รูป** (รถ 1 คันต่อ 1 รูป) ที่เก็บมาจาก **Wikimedia Commons** ซึ่งเป็นคลังสื่อเสรีของมูลนิธิวิกิมีเดีย ตามที่อาจารย์อนุญาตให้ใช้ข้อมูลจากอินเทอร์เน็ตได้ ทุกรูปผ่านการตรวจด้วยตาทีละรูป ย่อขนาด ตีกรอบป้ายทะเบียน และบันทึกที่มาไว้ในไฟล์ dataset_manifest.csv

ตารางที่ 3 สรุปชุดข้อมูล

หัวข้อ	ค่า
แหล่งที่มา	Wikimedia Commons (commons.wikimedia.org) — 250 จาก 250 รูป
จำนวนรูปที่ดาวน์โหลดทั้งหมด	416 รูป (~405 รูปไม่ซ้ำ ที่ใช้พิจารณา)
ผ่านเกณฑ์ (passed)	250 รูป ⟶ data/raw/
ไม่ผ่านเกณฑ์ (rejected)	166 รูป ⟶ data/raw/rejected/ (เก็บไว้เผื่อใช้เป็น negative examples)
ขนาดรูป	ด้านยาวสุดไม่เกิน 1280 พิกเซล (ส่วนใหญ่ 1280×960) JPEG คุณภาพ 85
คลาส	1 คลาส: license_plate
ผู้ถ่ายภาพ	19 ราย (มากที่สุด: M.rJirapat 119 รูป, Jirapat Chroenkeskij 43 รูป, nakhon100 34 รูป)
การแบ่งชุดข้อมูล	Test 50 รูป (แยกไว้) + Train pool 200 รูป (5-Fold ละ 40 รูป)

2.2 ทำไมเลือก Wikimedia Commons

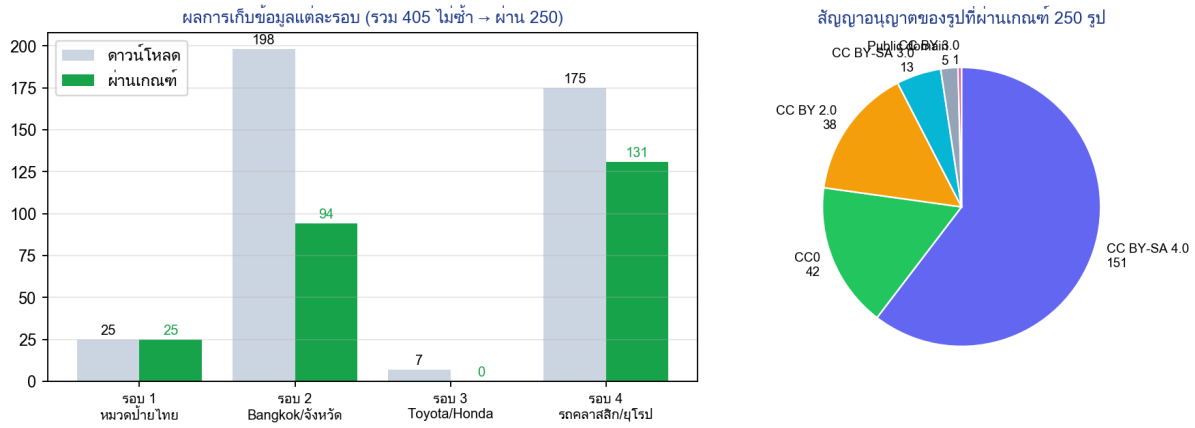
- **สัญญาอนุญาตเปิด:** รูปส่วนใหญ่ใช้สัญญาแบบ Creative Commons (CC BY-SA, CC BY, CC0) และ Public Domain ทำให้นำไปใช้ ครอป และดัดแปลงเพื่อการศึกษาได้ โดยต้องให้เครดิตผู้ถ่ายภาพ
- **มีหมวดหมู่จัดระเบียบ:** มีหมวดหมู่ เช่น Automobiles with license plates of Thailand, Automobiles in Bangkok ทำให้ค้นรูปรถไทยได้ตรงเป้าหมาย
- **มีข้อมูลกำกับครบ:** ทุกรูปมีหน้าข้อมูลบอกผู้ถ่าย สัญญาอนุญาต และต้นทาง ซึ่งนำมาบันทึกลง manifest เพื่อตรวจสอบย้อนหลังได้
- **คุณภาพเหมาะกับงาน:** เป็นรูปถ่ายรถบนท้องถนนจริง หลากหลายยี่ห้อ สีรถ สภาพแสง และจังหวัด

2.3 ขั้นตอนการเก็บข้อมูล 5 รอบ

การเก็บข้อมูลทำเป็นรอบ ๆ แล้วสังเกต "อัตราผ่านเกณฑ์"
ของแต่ละแหล่งเพื่อตัดสินใจว่าจะเก็บจากแหล่งไหนต่อ แหล่งไหนผ่านน้อยก็หยุดทันทีเพื่อประหยัดเวลา
ผลของแต่ละรอบแสดงในตารางที่ 4

ตารางที่ 4 ผลการเก็บข้อมูลแต่ละรอบ

รอบ	แหล่งที่ค้นหา	ดาวน์โหลด	ผ่าน	อัตราผ่าน
1	Automobiles with license plates of Thailand (หมวดที่คัดมาเพื่อป้ายโดยเฉพาะ)	25	25	100%
2	Automobiles in Bangkok + จังหวัดอื่น + Japanese spec (รูป car-spotting ทั่วไป)	198	94	47%
3	Toyota / Honda / Nissan by brand (ปนรูปโชว์รูม/แคตตาล็อกจำนวนมาก)	7	0	0% (หยุดทันที)
4	Isuzu / Mazda / BMW / Mercedes / Volvo / Suzuki by brand (ชุมชนนักสะสมรถคลาสสิก)	175	131	75% (ดีที่สุด)
รวม		~405 (ไม่ซ้ำ)	250	~62%



รูปที่ 1 ผลการเก็บข้อมูลแต่ละรอบ และสัดส่วนสัญญาอนุญาตของรูปที่ผ่านเกณฑ์

2.4 เกณฑ์การคัดเลือกรูป (Passed / Rejected)

ทุกรูป (416 รูป) ถูกตรวจด้วยตาจริงทีละรูป ไม่ใช่การสุ่มตรวจ โดยใช้เกณฑ์ต่อไปนี้
รูปที่ผ่านทุกข้อจะถูกตั้งสถานะ passed ส่วนรูปที่ไม่ผ่านจะถูกย้ายไปเก็บที่ rejected/ (ไม่ลบทิ้ง)

- มุมกล้อง: หน้าตรงหรือหลังตรงที่เห็นป้ายทะเบียน

- ระยะภาพ: เห็นตัวรถเต็มคัน ไม่ไกลเกินจนป้ายเล็กมาก
- เห็นป้ายทะเบียนไทยครบทั้งแผ่น (เห็นเป็นรูปทรงชัดเจน ไม่จำเป็นต้องอ่านตัวอักษรได้ทุกตัว เพราะเป้าหมายคือหาตำแหน่ง ไม่ใช่ OCR)
- เป็นรถ 1 คันหลักในเฟรม และเป็นรถที่ใช้งานจริงบนถนน (ไม่ใช่รถในโชว์รูมหรืองานแสดงรถ)

2.5 สัญญาอนุญาตและการให้เครดิต

ตารางที่ 5 สัญญาอนุญาตของรูปที่ผ่านเกณฑ์ 250 รูป

สัญญาอนุญาต	จำนวนรูป	หมายเหตุ
CC BY-SA 4.0	151	ต้องให้เครดิต และเผยแพร่งานดัดแปลงด้วยสัญญาเดียวกัน (share-alike)
CC0	42	สาธารณสมบัติโดยผู้เป็นเจ้าของสละสิทธิ์ ใช้ได้อิสระ
CC BY 2.0	38	ต้องให้เครดิตผู้ถ่ายภาพ
CC BY-SA 3.0	13	ต้องให้เครดิต + share-alike
Public domain	5	ใช้ได้อิสระ
CC BY 3.0	1	ต้องให้เครดิตผู้ถ่ายภาพ

ไม่มีรูปใดใช้สัญญาแบบ ND (No Derivatives) จึงสามารถครอปและดัดแปลงภาพได้ตามที่โครงงานต้องการ หากนำโครงงานไปเผยแพร่ ต้องให้เครดิตผู้ถ่ายภาพทุกคนตามรายการใน dataset_manifest.csv

2.6 บทเรียนสำคัญจากการเก็บข้อมูล

1. **หมวดหมู่ที่คัดมาเพื่อป้ายโดยเฉพาะให้ผลดีที่สุด (ผ่าน 100%) แต่มีจำนวนจำกัด** เพียง 25 ไฟล์ในหมวดทั้งหมด
2. **หมวด by-brand ของรถยอดนิยม (Toyota/Honda/Nissan) ปนรูปโชว์รูมจำนวนมาก** รถใหม่ยังไม่ติดป้ายจริง จึงผ่านเป็น 0% ต้องกรองชื่อไฟล์ก่อนดาวน์โหลด
3. **ชุมชนนักสะสมรถคลาสสิก/รถยุโรปให้ผลดีที่สุดในรอบกว้าง (75%)** เพราะเจ้าของมักถ่ายเต็มคันทั้งหน้าและท้าย ทำให้เห็นป้ายชัดโดยบังเอิญ
4. **รูปจากงานแสดงรถในห้างหรือพิพิธภัณฑ์ไม่มีป้ายจริง** ต้อง reject ทั้งหมด เพราะต่างจากสภาพใช้งานจริงบนถนน

5. **ป้ายที่ไม่ใช่มาตรฐานไทย (ตัวอักษรละติน) ถูก reject** แม้จะเป็นรถที่จดทะเบียนในไทย เพราะไม่ตรงเป้าหมายของระบบ
6. **ชุดข้อมูลมีรถซ้ำกันบ้าง** (หลายรูปของรถคันเดียวกันต่างมุม) จึงต้องจัดกลุ่มก่อนแบ่ง train/test (ดูหัวข้อ 2.9)

2.7 ตัวอย่างข้อมูลในชุด Dataset



รูปที่ 2 ตัวอย่างรูปในชุดข้อมูล พร้อมกรอบป้ายทะเบียนที่ตีไว้ (Ground Truth)

รูปในชุดข้อมูลมีความหลากหลายทั้งยี่ห้อ สีรถ ตำแหน่งป้าย และสภาพแสง ซึ่งช่วยให้โมเดลเรียนรู้ลักษณะ "ทั่วไป" ของป้ายทะเบียน ไม่ใช่จำเฉพาะรถบางคัน

2.8 การตีกรอบ (Labeling)

ผู้จัดทำตีกรอบสี่เหลี่ยมรอบป้ายทะเบียนของทั้ง 250 รูปด้วยเว็บแอป **makesense.ai** (ไม่ต้องติดตั้งโปรแกรม) กำหนดคลาสเดียวคือ license_plate แล้วส่งออกเป็น **YOLO format** ซึ่งเป็นไฟล์ข้อความ 1 ไฟล์ต่อ 1 รูป โดยแต่ละบรรทัดแทนหนึ่งกรอบ

ตัวอย่างไฟล์ label ของ plate_004.jpg (data/labels/plate_004.txt)

```
0 0.505115 0.552003 0.194373 0.119352
```

ตารางที่ 6 ความหมายของแต่ละค่าในไฟล์ label (YOLO format)

ตำแหน่ง	ค่าตัวอย่าง	ความหมาย
1	0	หมายเลขคลาส (0 = license_plate)
2	0.505115	พิกัด x ของจุดกึ่งกลางกรอบ (cx) เป็นสัดส่วนของความกว้างรูป
3	0.552003	พิกัด y ของจุดกึ่งกลางกรอบ (cy) เป็นสัดส่วนของความสูงรูป
4	0.194373	ความกว้างของกรอบ (w) เป็นสัดส่วนของความกว้างรูป
5	0.119352	ความสูงของกรอบ (h) เป็นสัดส่วนของความสูงรูป

การเก็บเป็นสัดส่วน 0–1 (normalized) ทำให้ label ใช้ได้ไม่ว่ารูปจะถูกย่อหรือขยายเป็นขนาดใดก็ตาม หลังตีกรอบเสร็จ ใช้สคริปต์ verify_labels.py ตรวจรูปแบบอัตโนมัติ และสร้าง contact sheet ให้ตรวจกรอบด้วยตาอีกชั้น ผลคือผ่านครบ 250/250 รูป

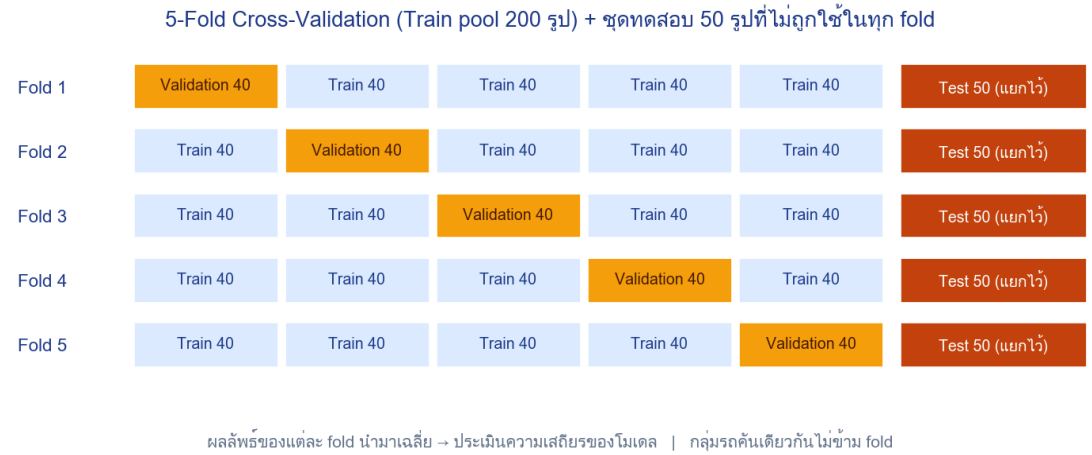


รูปที่ 3 Contact sheet สำหรับตรวจกรอบ label ด้วยตา (สุ่มจากชุดข้อมูล)

2.9 การแบ่งชุดข้อมูล (Data Split)

ข้อมูล 250 รูปถูกแบ่งเป็น **ชุดทดสอบ (Test) 50 รูป** ที่กันไว้ไม่ให้โมเดลเห็นเลยตลอดการฝึก และ **ชุดสอน (Train pool) 200 รูป** ซึ่งแบ่งเป็น 5 fold (fold ละ 40 รูป) สำหรับ 5-Fold Cross-Validation จุดสำคัญคือ **ไม่ให้รถคันเดียวกัน (หลายรูปหลายมุม) ตกอยู่คนละชุด** เพราะจะทำให้โมเดลเหมือน

"เห็นข้อสอบมาก่อน" และผลประเมินดีเกินจริง สคริปต์ split_folds.py จึงจัดกลุ่มรูปจากชื่อไฟล์ต้นฉบับใน manifest ก่อน แล้วแจกทั้งกลุ่มเข้าชุดเดียวกัน (ดูโค้ดหัวข้อ 4.3–4.4)



รูปที่ 4 การแบ่งชุดข้อมูลแบบ 5-Fold Cross-Validation และชุดทดสอบที่แยกไว้

2.10 dataset_manifest.csv: บันทึกที่มาของทุกรูป

ตารางที่ 7 คอลัมน์ในไฟล์ dataset_manifest.csv

คอลัมน์	ความหมาย
filename	ชื่อไฟล์ในโปรเจกต์ (เช่น plate_001.jpg)
source / author	แหล่งที่มา (Wikimedia Commons) และชื่อผู้ถ่ายภาพ
license	สัญญาอนุญาตของรูปนั้น (เช่น CC BY-SA 4.0)
source_url	ลิงก์หน้าไฟล์ต้นฉบับ ใช้ตรวจสอบย้อนหลังและใช้จัดกลุ่มรถคันเดียวกัน
sha256	ลายนิ้วมือของไฟล์ ใช้กันรูปซ้ำ
width / height	ขนาดรูปหลังย่อ
status	passed / rejected / pending_review

บทที่ 3 Algorithm: YOLO11 และเทคนิคที่เกี่ยวข้อง

บทนี้อธิบายทฤษฎีของอัลกอริทึมที่ใช้ในโครงงาน เริ่มจากแนวคิด Object Detection ไปจนถึงสถาปัตยกรรม YOLO11 วิธีการเรียนรู้ การวัดผล และข้อดี–ข้อเสีย
โดยภาพประกอบทั้งหมดในบทนี้ผู้จัดทำวาดขึ้นเองและใช้รูปจากข้อมูลจริงของโครงงาน
เพื่อให้อธิบายได้ตรงกับสิ่งที่ทำจริง (แหล่งอ้างอิงทางทฤษฎีอยู่ในบรรณานุกรมท้ายเล่ม)

3.1 Object Detection คืออะไร

งานด้านคอมพิวเตอร์วิชันแบ่งตามความละเอียดของคำตอบได้เป็นหลายระดับ **Image Classification** ตอบว่า "ในรูปนี้มีอะไร" (ได้คำตอบเดียวทั้งรูป) ส่วน **Object Detection** ตอบว่า "มีอะไรบ้าง และอยู่ตรงไหน" โดยให้ผลเป็นกรอบสี่เหลี่ยม (bounding box)
พร้อมชื่อคลาสและค่าความมั่นใจของแต่ละวัตถุ ส่วน Segmentation
ตอบละเอียดถึงระดับพิกเซลว่าพิกเซลไหนเป็นวัตถุใด

โครงงานนี้ต้องการ **ตำแหน่งของป้ายทะเบียน** เพื่อนำไปครอป จึงเหมาะกับ Object Detection
ซึ่งให้กรอบสี่เหลี่ยมที่พอดีกับการตัดภาพ โดยไม่จำเป็นต้องใช้ Segmentation
ที่ซับซ้อนและต้องตีเส้นขอบละเอียดกว่ามาก

ตารางที่ 8 เปรียบเทียบงานคอมพิวเตอร์วิชันที่เกี่ยวข้อง

งาน	คำถามที่ตอบ	ผลลัพธ์	เหมาะกับโครงงานนี้?
Image Classification	ในรูปมีอะไร	ชื่อคลาส 1 ค่า	ไม่ (ไม่รู้ตำแหน่ง)
Object Detection	มีอะไร อยู่ตรงไหน	กรอบ + คลาส + ความมั่นใจ	ใช่ (ใช้ในโครงงาน)
Semantic / Instance Segmentation	พิกเซลไหนเป็นวัตถุใด	มาสก์ระดับพิกเซล	เกินความจำเป็น
OCR	ตัวอักษรในภาพคืออะไร	ข้อความ	ขั้นตอนถัดไป (นอกขอบเขต)

3.2 วิวัฒนาการของ YOLO และแนวคิด One-Stage Detector

ก่อนหน้า YOLO ตัวตรวจจับยอดนิยมเป็นแบบ **Two-Stage** เช่น R-CNN, Fast R-CNN และ Faster R-CNN ซึ่งทำงาน 2 ขั้น คือ (1) เสนอบริเวณที่อาจมีวัตถุ (region proposal) แล้ว (2)
จำแนกและปรับกรอบของแต่ละบริเวณ วิธีนี้แม่นยำแต่ช้า เพราะต้องประมวลผลหลายร้อยบริเวณต่อรูป

ในปี 2016 Redmon และคณะเสนอ **YOLO (You Only Look Once)** ที่มองการตรวจจับเป็นปัญหา regression เดียว คือส่งรูปผ่านเครือข่ายประสาทเทียม **เพียงครั้งเดียว**
แล้วได้กรอบและคลาสทั้งหมดออกมาพร้อมกัน จึงเร็วมากจนใช้กับงานแบบ real-time ได้
หลังจากนั้นมีการพัฒนาต่อเนื่องหลายรุ่น

ตารางที่ 9 ไทม์ไลน์โดยสังเขปของตระกูล YOLO

รุ่น	ปี	จุดเด่น
YOLOv1	2016	แนวคิด one-stage แบ่งภาพเป็นตาราง ทำนายกรอบและคลาสในครั้งเดียว
YOLOv2 / v3	2017–2018	เพิ่ม anchor box, batch normalization, ทำนายหลายสเกล
YOLOv4 – v7	2020–2022	เทคนิคเพิ่มความแม่นยำ เช่น CSP, PAN, mosaic augmentation
YOLOv8	2023	Ultralytics: anchor-free, decoupled head, ใช้งานง่ายผ่าน Python API
YOLO11	2024	Ultralytics: บล็อก C3k2, โมดูลความสนใจ C2PSA, ประสิทธิภาพ/ขนาดโมเดลดีขึ้น

3.3 หลักการทำงานของ YOLO

แนวคิดดั้งเดิมของ YOLO คือแบ่งภาพเป็นตาราง (grid) เช่น S×S ช่อง **ช่องที่ "จุดกึ่งกลางของวัตถุ" ตกอยู่จะรับผิดชอบทำนายวัตถุนั้น** โดยแต่ละช่องจะทำนายค่าต่อไปนี้พร้อมกัน

- ตำแหน่งและขนาดกรอบ: (x, y, w, h)
- ความมั่นใจ (confidence) ว่ากรอบนี้มีวัตถุจริงและกรอบแม่นแค่ไหน
- ความน่าจะเป็นของแต่ละคลาส (ในโครงงานนี้มีคลาสเดียว)

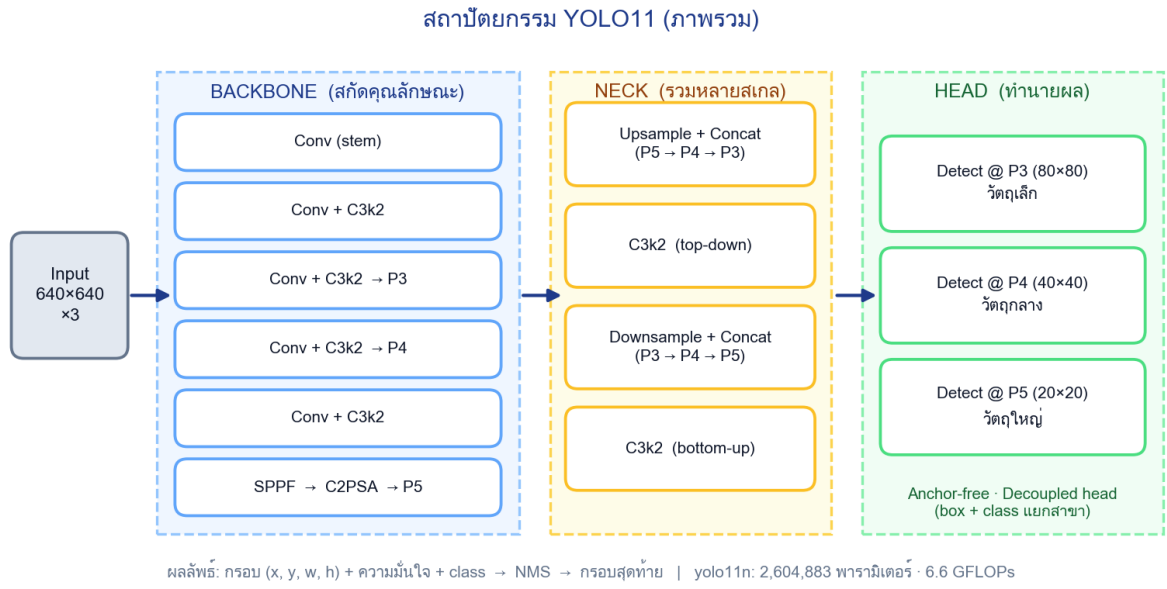


รูปที่ 5 แนวคิดตารางของ YOLO: ช่องที่มีจุดกึ่งกลางป้าย (สีส้ม) เป็นผู้ทำนายกรอบ (สีเขียว) ของป้ายทะเบียน

YOLO รุ่นใหม่ (v8 และ YOLO11) ไม่ได้ใช้ตารางขนาดเดียวอีกต่อไป แต่ทำนายบน **แผนที่คุณลักษณะ (feature map) 3 สเกล** คือ 80×80, 40×40 และ 20×20 (เมื่อ input 640×640) เพื่อรองรับวัตถุขนาดเล็ก กลาง และใหญ่ ตามลำดับ และทำนายแบบ **anchor-free**
คือทำนายระยะจากจุดบนแผนที่ไปยังขอบกรอบโดยตรง ไม่ต้องกำหนดกรอบตั้งต้น (anchor box) ไว้ล่วงหน้า

3.4 สถาปัตยกรรม YOLO11

YOLO11 ประกอบด้วย 3 ส่วนหลักที่ทำงานต่อกัน ดังรูปที่ 6



รูปที่ 6 สถาปัตยกรรม YOLO11 โดยสังเขป (Backbone ⟶ Neck ⟶ Head)

1) Backbone: สกัดคุณลักษณะจากภาพ

Backbone คือเครือข่ายคอนโวลูชัน (CNN) ที่รับภาพเข้ามาแล้วค่อย ๆ ลดขนาดภาพลงพร้อมเพิ่มความลึกของข้อมูล เพื่อสกัดคุณลักษณะจากระดับต่ำ (ขอบ มุม สี) ไปสู่ระดับสูง (รูปทรงสี่เหลี่ยมของป้าย ตัวอักษร) ส่วนประกอบสำคัญได้แก่

- **Conv:** ชั้นคอนโวลูชัน + Batch Normalization + ฟังก์ชันกระตุ้น SiLU ใช้ลดขนาดภาพ (stride 2)
- **C3k2:** บล็อกแบบ CSP (Cross Stage Partial) ที่แยกการไหลของคุณลักษณะเป็นสองสาย แล้วนำกลับมารวมกัน ช่วยลดการคำนวณซ้ำซ้อนแต่ยังรักษาความแม่นยำ — เป็นบล็อกหลักของ YOLO11 ที่พัฒนาจาก C2f ใน YOLOv8
- **SPPF:** Spatial Pyramid Pooling – Fast รวมข้อมูลจากบริเวณหลายขนาดเพื่อให้ได้บริบทกว้างขึ้นด้วยต้นทุนต่ำ
- **C2PSA:** โมดูลที่ใช้กลไก Attention แบบ Position-Sensitive ให้เครือข่ายโฟกัสบริเวณสำคัญของภาพ (เพิ่มใน YOLO11)

2) Neck: รวมคุณลักษณะหลายสเกล

Neck ทำหน้าที่ผสานคุณลักษณะจากหลายระดับของ Backbone ด้วยโครงสร้างแบบ Feature Pyramid (FPN) ร่วมกับ Path Aggregation (PAN) คือส่งข้อมูลความหมายระดับสูงจากชั้นลึกลงมา (top-down) และส่งรายละเอียดตำแหน่งจากชั้นตื้นขึ้นไป (bottom-up) ทำให้ได้แผนที่คุณลักษณะที่ทั้ง "รู้ว่าเป็นอะไร" และ "รู้ว่าอยู่ตรงไหน" ในทุกสเกล

3) Head: ทำนายผลลัพธ์

Head ของ YOLO11 เป็นแบบ **Decoupled Head** คือแยกสาขาการทำนาย "ตำแหน่งกรอบ" ออกจากสาขาการทำนาย "คลาส" ซึ่งฝึกง่ายและแม่นกว่าการใช้สาขาร่วมกัน และเป็นแบบ **Anchor-free** ตามที่อธิบายในหัวข้อ 3.3 ผลลัพธ์ดิบจะมีกรอบจำนวนมากซ้อนทับกัน ซึ่งต้องกรองด้วยขั้นตอน NMS (หัวข้อ 3.7)

ตารางที่ 10 ข้อมูลโมเดล YOLO11n ที่ใช้ในโครงงาน

รายการ	ค่า
รุ่นที่ใช้	yolo11n (nano) — รุ่นเล็กที่สุดของตระกูล YOLO11
จำนวนชั้น / พารามิเตอร์	182 ชั้น / 2,604,883 พารามิเตอร์ (หลังปรับเป็น 1 คลาส)
ปริมาณการคำนวณ	6.6 GFLOPs ต่อภาพ 640×640
ขนาดไฟล์โมเดลที่ฝึกแล้ว	ประมาณ 5.2 MB (models/best.pt)
น้ำหนักตั้งต้น (pretrained)	yolo11n.pt ที่ฝึกบนชุดข้อมูล COCO (80 คลาส)
Input size	640×640 พิกเซล

รุ่น nano เหมาะกับโครงงานนี้เพราะงานมีคลาสเดียวและลักษณะวัตถุชัดเจน (สี่เหลี่ยมแบนมีตัวอักษร) ไม่จำเป็นต้องใช้โมเดลใหญ่ ทำให้ฝึกเร็ว รันบนเครื่องส่วนตัวได้ และข้อมูลฝึกแค่ 200 รูปก็ไม่เสี่ยง overfit มากเท่ากับโมเดลใหญ่

3.5 Loss Function: โมเดลเรียนรู้จากอะไร

ระหว่างฝึก โมเดลจะเปรียบเทียบสิ่งที่ทำนายกับกรอบจริง (ground truth) แล้วคำนวณ "ค่าความผิดพลาด" เพื่อปรับน้ำหนักของเครือข่ายด้วย gradient descent โดย YOLO11 ใช้ loss 3 ส่วนรวมกัน

ตารางที่ 11 องค์ประกอบของ Loss ใน YOLO11 (ตามที่บันทึกในผลการฝึก)

Loss	วัดอะไร	วิธีคำนวณโดยสังเขป
box_loss	กรอบที่ทำนายห่างจากกรอบจริงแค่ไหน	CIoU Loss: พิจารณาพื้นที่ซ้อนทับ (IoU) + ระยะห่างจุดกึ่งกลาง + ความต่างของสัดส่วนกว้าง/สูง
cls_loss	ทำนายคลาสผิดแค่ไหน	Binary Cross-Entropy

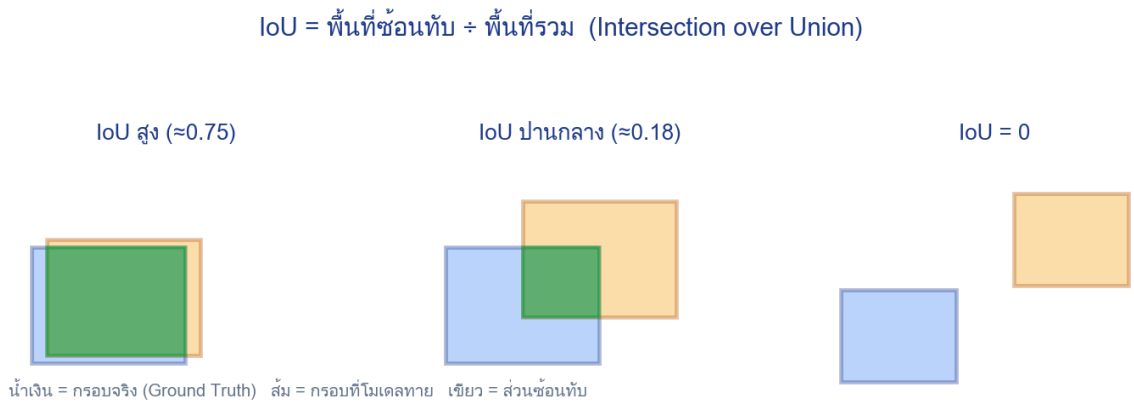
Loss	วัดอะไร	วิธีคำนวณโดยสังเขป
dfl_loss	ความแม่นของการทำนายขอบกรอบ	Distribution Focal Loss: ทำนายขอบเป็นการกระจายความน่าจะเป็น แล้วเรียนรู้ให้แหลมตรงตำแหน่งจริง

นอกจากนี้ยังใช้ **Task-Aligned Assigner** เลือกว่ากรอบที่ทำนายอันไหนควร "จับคู่"
กับกรอบจริงอันไหนระหว่างฝึก โดยพิจารณาทั้งคะแนนคลาสและคุณภาพตำแหน่งร่วมกัน

3.6 IoU (Intersection over Union)

IoU เป็นตัวชี้วัดความทับซ้อนระหว่างกรอบสองกรอบ ใช้ทั้งในการฝึก (loss), การทำ NMS
และการประเมินว่าตรวจจับ "ถูกตำแหน่ง" หรือไม่
คำนวณจากพื้นที่ส่วนที่ซ้อนกันหารด้วยพื้นที่รวมของกรอบทั้งสอง

IoU = พื้นที่ส่วนซ้อนทับ ÷ (พื้นที่กรอบ A + พื้นที่กรอบ B − พื้นที่ส่วนซ้อนทับ)



รูปที่ 7 ตัวอย่างค่า IoU: ซ้อนทับมาก ปานกลาง และไม่ซ้อนทับเลย

ค่า IoU อยู่ระหว่าง 0 (ไม่ซ้อนกันเลย) ถึง 1 (ซ้อนกันสมบูรณ์) ในโครงงานนี้ใช้เกณฑ์ **IoU ≥ 0.5**
ในการนับว่ากรอบที่โมเดลทำนาย "ตรงกับป้ายจริง" และใช้ IoU = 0.45 เป็นเกณฑ์ของ NMS

3.7 NMS (Non-Maximum Suppression)

เนื่องจากโมเดลทำนายบนหลายตำแหน่งและหลายสเกล วัตถุเดียวกันมักได้กรอบซ้อนกันหลายกรอบ NMS
ใช้กรองให้เหลือกรอบที่ดีที่สุดเพียงกรอบเดียวต่อวัตถุ มีขั้นตอนดังนี้

1. ทิ้งกรอบที่ความมั่นใจต่ำกว่าเกณฑ์ (confidence threshold)
2. เรียงกรอบที่เหลือจากความมั่นใจสูงไปต่ำ

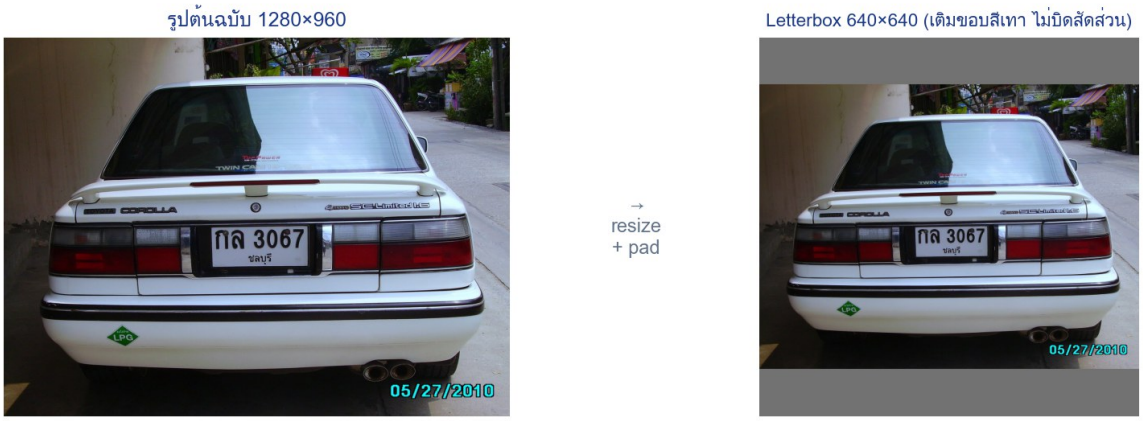
3. เลือกกรอบที่มั่นใจสูงสุดเก็บไว้
4. คำนวณ IoU ระหว่างกรอบที่เก็บไว้กับกรอบอื่นที่เหลือ กรอบใดมี IoU สูงกว่าเกณฑ์ (0.45) จะถูกตัดทิ้งเพราะถือว่าเป็นวัตถุเดียวกัน
5. ทำซ้ำข้อ 3–4 กับกรอบที่เหลือจนครบ



รูปที่ 8 ก่อนและหลังทำ NMS: กรอบซ้อนกันหลายกรอบเหลือเพียงกรอบที่มั่นใจที่สุด (ภาพประกอบจำลองเพื่ออธิบาย)

3.8 Preprocessing: Letterbox Resize

โมเดลรับภาพขนาดคงที่ 640×640 แต่รูปจริงมีสัดส่วนต่างกัน (เช่น 1280×960) หากยืดภาพให้เป็นสี่เหลี่ยมจัตุรัสตรง ๆ ป้ายทะเบียนจะบิดเบี้ยว YOLO จึงใช้ **letterbox** คือย่อภาพโดยคงสัดส่วนเดิมให้ด้านยาวเท่ากับ 640 แล้วเติมขอบสีเทา (ค่า 114) ให้เต็มกรอบ ทำให้รูปทรงของป้ายไม่ผิดเพี้ยน ไลบรารี Ultralytics ทำขั้นตอนนี้ให้อัตโนมัติ และแปลงพิกัดกรอบกลับเป็นพิกัดบนรูปต้นฉบับให้เองด้วย



รูปที่ 9 การทำ Letterbox: ย่อรูปโดยคงสัดส่วน แล้วเติมขอบเทาให้เป็น 640×640

3.9 Transfer Learning

การฝึกโมเดลจากศูนย์ต้องใช้รูปจำนวนมาก (หลักหมื่นถึงล้านรูป) แต่โครงงานนี้มีข้อมูลเพียง 200 รูปสำหรับฝึก จึงใช้ **Transfer Learning** คือเริ่มจากน้ำหนัก yolo11n.pt ที่ฝึกมาแล้วบนชุดข้อมูล COCO (วัตถุทั่วไป 80 คลาส) ซึ่งโมเดลได้เรียนรู้คุณลักษณะพื้นฐาน เช่น ขอบ มุม พื้นผิว รูปทรง ไว้แล้ว จากนั้นปรับ (fine-tune) ให้เชี่ยวชาญเฉพาะงานตรวจจับป้ายทะเบียน ทำให้ลู่เข้าเร็วและได้ความแม่นยำสูงแม้ข้อมูลน้อย

3.10 Data Augmentation

Data Augmentation คือการสร้างความหลากหลายให้ข้อมูลฝึกโดยปรับแต่งภาพแบบสุ่มระหว่างฝึก เพื่อลดการ overfit Ultralytics เปิดใช้เทคนิคเหล่านี้ตามค่าเริ่มต้น เช่น **Mosaic** (นำ 4 รูปมาต่อกันเป็นภาพเดียว), พลิกซ้าย-ขวา, ปรับสี/ความสว่าง (HSV), ย่อ-ขยาย และเลื่อนตำแหน่ง พร้อมปิด mosaic ใน 10 epoch สุดท้าย (close_mosaic = 10) เพื่อให้โมเดลปรับตัวกับภาพจริงปกติก่อนจบ



รูปที่ 10 ตัวอย่าง batch ระหว่างฝึก (fold 3) จะเห็นการต่อภาพแบบ Mosaic และกรอบ label

3.11 ตัวชี้วัดการประเมินผล (Evaluation Metrics)

การประเมินผลของ Object Detection ต้องนิยามว่าอะไรคือ "ตรวจถูก" โดยใช้ IoU: กรอบที่ทำนายที่มี IoU กับกรอบจริงตั้งแต่เกณฑ์ที่กำหนด (เช่น 0.5) ถือเป็นการตรวจถูก (True Positive, TP) กรอบที่ทำนายแต่ไม่ตรงกับของจริงคือ False Positive (FP) และป้ายจริงที่โมเดลหาไม่เจอคือ False Negative (FN)

ตารางที่ 12 นิยามตัวชี้วัดที่ใช้ในโครงงาน

ตัวชี้วัด	สูตร / ความหมาย	ตีความ
Precision	TP ÷ (TP + FP)	กรอบที่โมเดลบอกว่าเป็นป้าย ถูกจริงกี่ %
Recall	TP ÷ (TP + FN)	ป้ายทั้งหมดในรูป หาเจอกี่ %

ตัวชี้วัด	สูตร / ความหมาย	ตีความ
AP (Average Precision)	พื้นที่ใต้กราฟ Precision–Recall	สรุปคุณภาพการตรวจจับของ 1 คลาสเป็นตัวเลขเดียว
mAP@0.5	ค่าเฉลี่ย AP ทุกคลาส เมื่อนับถูกที่ IoU ≥ 0.5	คะแนนรวมหลัก (โครงงานมี 1 คลาส จึงเท่ากับ AP ของป้าย)
mAP@0.5:0.95	เฉลี่ย mAP ที่ IoU 0.5 ถึง 0.95	เข้มงวดกว่า ดูความแม่นยำของตำแหน่งกรอบ
Crop Success Rate (กำหนดเอง)	จำนวนรูปที่ครอปถูกตำแหน่ง (IoU ≥ 0.5) ÷ จำนวนรูปทดสอบ	วัดผลลัพธ์ปลายทางของระบบจริง คือ ครอปได้ป้ายถูกกี่ %

3.12 K-Fold Cross-Validation

การแบ่งข้อมูลฝึกและตรวจเพียงครั้งเดียวมีความเสี่ยงว่าผลจะขึ้นกับว่า
"บังเอิญสุ่มได้รูปง่ายหรือยากเข้าชุดตรวจ" **K-Fold Cross-Validation** แก้ปัญหานี้โดยแบ่งข้อมูลเป็น K ส่วนเท่า ๆ กัน (ในโครงงานนี้ K = 5) แล้วฝึกโมเดล K รอบ แต่ละรอบใช้ 1 ส่วนเป็นชุดตรวจ (validation) และอีก K−1 ส่วนเป็นชุดสอน สลับกันจนทุกส่วนได้เป็นชุดตรวจ
ผลเฉลี่ยของทุกรอบจึงสะท้อนความสามารถของโมเดลได้เสถียรกว่า และค่าความแปรปรวนระหว่าง fold บอกได้ว่าโมเดลไวต่อข้อมูลมากน้อยเพียงใด (ดูรูปที่ 4)

3.13 การครอปภาพด้วย OpenCV

เมื่อได้กรอบ (x0, y0, x1, y1) จาก YOLO แล้ว การครอปคือการตัดอาร์เรย์ภาพตามพิกัดนั้น (image[y0:y1, x0:x1]) โครงงานเพิ่มขอบอีก 3 พิกเซลทุกด้านเพื่อกันไม่ให้ตัดขอบป้ายหรือตัวอักษรชิดขอบขาด (โดยไม่ให้เกินขอบรูป) แล้วปรับเป็นขนาดมาตรฐาน 400×200 ด้วย **Bicubic interpolation (cv2.INTER_CUBIC)** ซึ่งคำนวณพิกเซลใหม่จากพิกเซลข้างเคียง 16 จุด ให้ภาพที่นุ่มและคมกว่าวิธี nearest neighbor หรือ bilinear เมื่อต้องขยายป้ายเล็กให้ใหญ่ขึ้น

3.14 ข้อดีและข้อเสียของ YOLO11

ตารางที่ 13 ข้อดี–ข้อเสียของ YOLO11 (เมื่อใช้กับงานตรวจจับป้ายทะเบียน)

ข้อดี	ข้อเสีย / ข้อจำกัด
เร็ว: ประมวลผลรูปครั้งเดียว (one-stage) ตรวจรูปใน ~1.4–1.5 วินาทีบนเครื่องส่วนตัวรวมเวลาโหลดโมเดล (โหมดหลายรูปโหลดโมเดลครั้งเดียว รูปถัดไปใช้เวลาไม่ถึง 1 วินาที)	**ต้องมีข้อมูลที่ตีกรอบ:** ต้องเสียเวลาเก็บและตีกรอบรูปจำนวนหนึ่ง (โครงงานนี้ 250 รูป)
ขนาดเล็ก: yolo11n มี 2.6 ล้านพารามิเตอร์ ไฟล์ ~5 MB นำไปติดตั้งง่าย	**ไวต่อโดเมนที่ไม่เคยเห็น:** เช่น ป้ายสีแดง มุมเฉียง แสงน้อย ที่มีตัวอย่างน้อยอาจพลาด (เจอจริงใน plate_110)
ใช้งานง่าย: Ultralytics มี API ฝึก/ประเมิน/ตรวจจับในไม่กี่บรรทัด พร้อมกราฟผลลัพธ์อัตโนมัติ	**ให้ผลเป็นกรอบเท่านั้น:** ไม่ได้อ่านตัวอักษรบนป้าย ต้องต่อด้วย OCR หากต้องการเลขทะเบียน
Transfer Learning ได้ผลดี: ข้อมูลฝึกเพียง 200 รูปก็ได้ mAP@0.5 ≈ 0.99 บน CV	**วัตถุเล็กมากหรือซ้อนทับหนาแน่นทำได้ยากกว่า:** ป้ายที่เล็กเกินไปในภาพอาจตรวจไม่เจอ
Anchor-free: ไม่ต้องจูนกรอบตั้งต้น ลดการตั้งค่าที่ต้องปรับเอง	**ต้องจูน threshold:** ค่า confidence/IoU ที่ไม่เหมาะทำให้พลาดป้ายหรือได้กรอบซ้ำ
ปรับขยายได้: มีรุ่น n/s/m/l/x เลือกตามความต้องการด้านความเร็วกับความแม่นยำ	**Black box:** อธิบายเหตุผลการตัดสินใจของโมเดลได้ยาก ต้องพึ่งการวิเคราะห์ error ภายหลัง

3.15 เปรียบเทียบกับวิธีอื่นในการหาป้ายทะเบียน

ตารางที่ 14 เปรียบเทียบแนวทางการหาตำแหน่งป้ายทะเบียน

แนวทาง	หลักการ	ข้อดี	ข้อเสีย
Image Processing + กฎ (edge / contour / สี)	หาขอบสี่เหลี่ยมและกรองด้วยสัดส่วน/สี	ไม่ต้องฝึกโมเดล เบามาก	พังง่ายเมื่อแสง มุม หรือฉากหลังเปลี่ยน ต้องจูนกฎเอง
Haar Cascade / HOG + SVM	ใช้คุณลักษณะที่ออกแบบด้วยมือ + ตัวจำแนกแบบดั้งเดิม	เร็ว ใช้ข้อมูลน้อยกว่า deep learning	แม่นยำจำกัด ทนต่อการเปลี่ยนแปลงต่ำ

แนวทาง	หลักการ	ข้อดี	ข้อเสีย
Faster R-CNN (two-stage)	เสนอบริเวณก่อน แล้วจำแนกและปรับกรอบ	แม่นยำสูง	ช้า โมเดลใหญ่ ใช้งานจริงยากกว่า
SSD	ตรวจจับ one-stage หลายสเกล	เร็ว	ความแม่นยำกับวัตถุเล็กสู้ YOLO รุ่นใหม่ไม่ได้
YOLO11n (ที่ใช้)	one-stage anchor-free หลายสเกล	เร็ว เล็ก แม่นยำ ใช้ง่าย	ต้องมีข้อมูลตีกรอบ ไวต่อโดเมนที่ไม่เคยเห็น

บทที่ 4 Code: ตัวอย่างโค้ดสำคัญของโครงงาน 10 ส่วน

บทนี้ยกตัวอย่างโค้ดสำคัญ 10 ส่วนจากโปรเจกต์จริง เรียงตามลำดับของ pipeline ตั้งแต่เตรียมข้อมูล แบ่งข้อมูล ฝึกโมเดล ตรวจจับและครอป ประเมินผล ไปจนถึงรับ feedback จากผู้ใช้ แต่ละส่วนระบุไฟล์และบรรทัดที่อยู่ในโปรเจกต์ (ตัดย่อเฉพาะส่วนที่เกี่ยวข้อง)

4.1 กันรูปซ้ำด้วย SHA-256 (ingest_images.py)

ไฟล์ src/data_prep/ingest_images.py (เริ่มบรรทัดที่ 31)

```
def sha256_of_file(path: Path) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(8192), b""):
            h.update(chunk)
    return h.hexdigest()
```

ฟังก์ชันนี้อ่านไฟล์ทีละ 8 KB แล้วคำนวณค่าแฮช SHA-256 ของไฟล์ทั้งไฟล์ ได้เป็นสตริง 64 ตัวอักษรที่ไม่ซ้ำกันสำหรับเนื้อหาไฟล์ต่างกัน

ความสำคัญ: ตอนนำรูปเข้าระบบ จะนำค่าแฮชไปเทียบกับ manifest ถ้าซ้ำถือว่ารูปเดิม ข้ามทันที ช่วยกันรูปซ้ำที่ดาวน์โหลดจากหลายหมวด ซึ่งจะทำให้ข้อมูลเสียสมดุลและผลประเมินคลาดเคลื่อน การอ่านทีละก้อนทำให้ใช้หน่วยความจำน้อยแม้ไฟล์ใหญ่

4.2 ย่อรูปและบันทึกเป็นมาตรฐานเดียวกัน (ingest_images.py)

ไฟล์ src/data_prep/ingest_images.py (เริ่มบรรทัดที่ 67)

```
def resize_and_save(src_path: Path, dst_path: Path) -> tuple[int, int]:
    im = Image.open(src_path).convert("RGB")
    w, h = im.size
    scale = MAX_SIDE / max(w, h)
    if scale < 1:
        im = im.resize((int(w * scale), int(h * scale)), Image.LANCZOS)
    im.save(dst_path, "JPEG", quality=JPEG_QUALITY, optimize=True)
    return im.size
```

ฟังก์ชันแปลงรูปเป็น RGB แล้วคำนวณ scale = 1280 ÷ ด้านยาวสุด ถ้ารูปใหญ่กว่า 1280 จึงย่อด้วยวิธี LANCZOS (คุณภาพสูง) คงสัดส่วนเดิม แล้วบันทึกเป็น JPEG คุณภาพ 85

ความสำคัญ: รูปจากอินเทอร์เน็ตมีขนาดและฟอร์แมตต่างกันมาก การปรับให้เป็นมาตรฐานเดียว (ด้านยาวไม่เกิน 1280, JPEG) ลดพื้นที่จัดเก็บและเวลาโหลดตอนฝึก โดยไม่เสียรายละเอียดป้ายที่จำเป็น เงื่อนไข scale < 1 ทำให้ไม่ขยายรูปเล็กจนภาพเบลอ

4.3 จัดกลุ่มรถคันเดียวกันจากชื่อไฟล์ต้นฉบับ (split_folds.py)

ไฟล์ src/data_prep/split_folds.py (เริ่มบรรทัดที่ 38)

```
def extract_group_key(source_url: str) -> str:
    """ดึง 'ชื่อรถ' จาก URL ต้นฉบับ เพื่อจัดกลุ่มรูปที่เป็นรถคันเดียวกัน"""
    name = source_url.split("File:")[-1] if "File:" in source_url else source_url
    name = urllib.parse.unquote(name)
    name = re.sub(r"\.(jpg|jpeg|png|JPG|JPEG|PNG)$", "", name)
    name = re.sub(r"\s*\(\d{6,}\)\s*$", "", name)  # flickr id ท้ายชื่อ
    name = re.sub(r"[\s_]*(front|rear|side|left|right|cropped|\d{1,2})[\s_]*$", "",
        name, flags=re.IGNORECASE)
    name = re.sub(r"\s*\([^)]*\)\s*$", "", name)
    return name.strip().lower()
```

ฟังก์ชันนี้ดึงชื่อไฟล์จาก source_url (เช่น File:1984-1986_Toyota_Corona_(AT150)_GL_(22-05-2017)_06.jpg) แล้วใช้ regular expression ตัดส่วนที่ต่างกันระหว่างรูปของรถคันเดียวกันออก เช่น นามสกุลไฟล์ รหัสตัวเลขท้ายชื่อ คำบอกมุม (front, rear, side) และเลขลำดับ แล้วแปลงเป็นตัวพิมพ์เล็ก

ความสำคัญ: ผลลัพธ์คือ "กุญแจกลุ่ม" ของรถแต่ละคัน รูปที่ได้กุญแจเดียวกันจะถูกแบ่งไปชุดเดียวกันเสมอ ป้องกัน data leakage (รถคันเดียวกันอยู่ทั้งชุดสอนและชุดทดสอบ)
ซึ่งเป็นสาเหตุที่ทำให้ผลประเมินดีเกินจริง

4.4 แจกกลุ่มรถเข้า 5 Fold อย่างสมดุล (split_folds.py)

ไฟล์ src/data_prep/split_folds.py (เริ่มบรรทัดที่ 121)

```
    fold_assign = [[] for _ in range(N_FOLDS)]
    fold_counts = [0] * N_FOLDS
    target_per_fold = len(train_pool_files) / N_FOLDS


    for key, files in pool_groups_shuffled:
        # ใส่กลุ่มนี้ลง fold ที่มีรูปน้อยที่สุดตอนนี้ (ทำให้สมดุลใกล้เคียงกัน)
        min_fold = min(range(N_FOLDS), key=lambda i: fold_counts[i])
        fold_assign[min_fold].extend(files)
        fold_counts[min_fold] += len(files)


    print("จำนวนรูปต่อ fold:", fold_counts)
```

โค้ดสุ่มลำดับของกลุ่มรถ (seed = 42 เพื่อให้ผลซ้ำได้) แล้วไล่แจกทีละกลุ่มเข้า fold ที่ตอนนั้นมีจำนวนรูป **น้อยที่สุด** (greedy balancing) จากนั้น fold ที่ k ใช้เป็นชุดตรวจ ส่วน fold ที่เหลือรวมกันเป็นชุดสอน

ความสำคัญ: แจกเป็น "กลุ่ม" ไม่ใช่รายรูป จึงไม่ทำให้รถคันเดียวกันข้าม fold และการเลือก fold ที่เล็กที่สุดทำให้แต่ละ fold มีขนาดใกล้เคียงกัน (ประมาณ 40 รูป) ผลของแต่ละ fold จึงเทียบกันได้อย่างยุติธรรม

4.5 สร้างไฟล์ data.yaml ให้แต่ละ fold (split_folds.py)

ไฟล์ src/data_prep/split_folds.py (เริ่มบรรทัดที่ 83)

```
def write_data_yaml(path: Path, train_dir: Path, val_dir: Path):
    content = (
        f"train: {train_dir.resolve()}\n"
        f"val: {val_dir.resolve()}\n"
        f"nc: 1\n"
        f"names: ['license_plate']\n"
    )
    path.write_text(content, encoding="utf-8")
```

Ultralytics ต้องการไฟล์ data.yaml เพื่อบอกที่อยู่ของรูปชุดสอน/ชุดตรวจ จำนวนคลาส (nc) และชื่อคลาส ฟังก์ชันนี้เขียนไฟล์ดังกล่าวให้อัตโนมัติสำหรับแต่ละ fold โดยใช้พาธแบบ absolute

ความสำคัญ: ทำให้การฝึกครบ 5 fold ทำได้ด้วยสคริปต์เดียวโดยไม่ต้องแก้ไฟล์ตั้งค่าด้วยมือ และใช้ symlink ชี้ไปยังรูปเดิม ไม่ต้องคัดลอกรูปซ้ำจึงประหยัดพื้นที่

4.6 ฝึกและประเมินโมเดลแต่ละ Fold (train_yolo11.py)

ไฟล์ src/train/train_yolo11.py (เริ่มบรรทัดที่ 64)

```
        model = YOLO(args.model)
        run_name = f"fold_{fold}"
        train_result = model.train(
            data=str(data_yaml),
            epochs=args.epochs,
            imgsz=args.imgsz,
            batch=args.batch,
            device=args.device,
            project=str(OUT_DIR / "runs"),
            name=run_name,
            exist_ok=True,
            verbose=not args.smoke,
            plots=not args.smoke,
        )


        metrics = model.val(
            data=str(data_yaml),
            device=args.device,
            project=str(OUT_DIR / "runs"),
            name=f"{run_name}_val",
            plots=not args.smoke,
        )
        precision = float(metrics.box.mp)
```

ในแต่ละ fold โค้ดโหลด yolo11n.pt (Transfer Learning) แล้วเรียก model.train() ด้วยพารามิเตอร์หลัก คือ epochs = 100, imgsz = 640, batch = 16, device = mps จากนั้นเรียก model.val() เพื่อวัด Precision, Recall และ mAP@0.5 บนชุดตรวจของ fold นั้น

ความสำคัญ: นี่คือหัวใจของการเรียนรู้ ผลของทุก fold ถูกรวมเป็น cv_summary.csv และคัดเลือก fold ที่มี mAP@0.5 สูงสุดไปเป็น models/best.pt อัตโนมัติ (เป็น fold 3) การสร้างโมเดลใหม่ทุก fold (YOLO(args.model) ภายในลูป) ทำให้แต่ละ fold เริ่มจากน้ำหนักเดียวกัน ไม่ปนข้อมูลข้าม fold

4.7 ตรวจจับป้ายและเลือกกรอบหลัก (predict_and_crop.py)

ไฟล์ src/inference/predict_and_crop.py (เริ่มบรรทัดที่ 83)

```
    t0 = time.perf_counter()
    if model is None:
        model = YOLO(str(weights_path))
    results = model.predict(source=str(image_path), conf=conf, iou=iou,
        device=resolve_device(device), verbose=False)
    boxes = results[0].boxes
    elapsed_ms = (time.perf_counter() - t0) * 1000


    if boxes is None or len(boxes) == 0:
        return None, "ไม่พบป้ายทะเบียน", {
            "num_detections": 0, "box_xyxy": None, "confidence": None, "elapsed_ms":
                elapsed_ms,
        }


    # เลือกกรอบใหญ่ที่สุด (รถคันหลัก) ถ้าเจอหลายป้าย
    areas = (boxes.xyxy[:, 2] - boxes.xyxy[:, 0]) * (boxes.xyxy[:, 3] - boxes.xyxy[:,
        1])
    best_idx = int(areas.argmax())
    best_box = boxes.xyxy[best_idx].tolist()
    best_conf = float(boxes.conf[best_idx])
```

โค้ดเรียก model.predict() พร้อมค่า conf (ทิ้งกรอบที่ความมั่นใจต่ำ) และ iou (เกณฑ์ NMS) ถ้าไม่เหลือกรอบจะคืนค่า "ไม่พบป้ายทะเบียน" ถ้าเจอหลายกรอบ จะคำนวณพื้นที่ (กว้าง × สูง) ของทุกกรอบ แล้วเลือกกรอบที่ **ใหญ่ที่สุด** ด้วย argmax ฟังก์ชันรับโมเดลที่โหลดไว้แล้วผ่านพารามิเตอร์ model ได้ (ถ้าไม่ส่งมาจะโหลดเอง) ทำให้โหมดหลายรูปและโหมดโต้ตอบโหลดโมเดลเพียงครั้งเดียว และเลือกอุปกรณ์ประมวลผลอัตโนมัติผ่านฟังก์ชัน resolve_device() (CUDA ⟶ MPS ⟶ CPU)

ความสำคัญ: กฎ "เลือกป้ายใหญ่สุด" ตัดสินใจว่าป้ายไหนเป็นของรถคันหลักในภาพ (รถคันหลักมักอยู่ใกล้กล้องและป้ายจึงใหญ่กว่า) นอกจากนี้ยังจับเวลา (elapsed_ms) เพื่อแสดงความเร็วในการตรวจจับ

4.8 ครอปป้ายด้วย OpenCV (crop_utils.py)

ไฟล์ src/crop/crop_utils.py (เริ่มบรรทัดที่ 12)

```
def crop_plate(image: np.ndarray, xyxy_box: tuple[float, float, float, float]) ->
    np.ndarray:
    """
    ครอปป้ายทะเบียนจากรูปต้นฉบับตามกรอบที่ตรวจจับได้ (พิกเซลจริง ไม่ใช่ normalized)
    ขยายขอบ PAD_PX พิกเซล แต่ไม่เกินขอบรูปต้นฉบับ แล้ว resize เป็น OUTPUT_SIZE
    """
    h, w = image.shape[:2]
    x0, y0, x1, y1 = xyxy_box


    x0 = max(0, int(round(x0)) - PAD_PX)
    y0 = max(0, int(round(y0)) - PAD_PX)
    x1 = min(w, int(round(x1)) + PAD_PX)
    y1 = min(h, int(round(y1)) + PAD_PX)


    if x1 <= x0 or y1 <= y0:
```

```
        raise ValueError(f"กรอบไม่ถูกต้อง: ({x0},{y0},{x1},{y1}) บนรูปขนาด {w}x{h}")


    cropped = image[y0:y1, x0:x1]
    resized = cv2.resize(cropped, OUTPUT_SIZE, interpolation=cv2.INTER_CUBIC)
    return resized
```

ฟังก์ชันแปลงพิกัดกรอบเป็นจำนวนเต็ม ขยายขอบ 3 พิกเซลทุกด้านด้วย max()/min()
เพื่อไม่ให้พิกัดเกินขอบรูป ตรวจว่ากรอบถูกต้องจริง (x1 > x0 และ y1 > y0) แล้วตัดภาพด้วย slicing
และปรับเป็น 400×200 ด้วย INTER_CUBIC

ความสำคัญ: เป็นขั้นตอนที่สร้าง "ผลลัพธ์สุดท้าย" ของระบบ
การกันขอบเกินรูปและการตรวจความถูกต้องของกรอบป้องกันข้อผิดพลาดรันไทม์
ส่วนการขยายขอบทำให้ตัวอักษรชิดขอบป้ายไม่ถูกตัดขาด

4.9 คำนวณ IoU เพื่อประเมินอัตราครอปสำเร็จ (evaluate_test_set.py)

ไฟล์ src/train/evaluate_test_set.py (เริ่มบรรทัดที่ 41)

```
def iou(box_a, box_b):
    ax0, ay0, ax1, ay1 = box_a
    bx0, by0, bx1, by1 = box_b
    inter_x0 = max(ax0, bx0)
    inter_y0 = max(ay0, by0)
    inter_x1 = min(ax1, bx1)
    inter_y1 = min(ay1, by1)
    inter_w = max(0.0, inter_x1 - inter_x0)
    inter_h = max(0.0, inter_y1 - inter_y0)
    inter_area = inter_w * inter_h
    area_a = (ax1 - ax0) * (ay1 - ay0)
    area_b = (bx1 - bx0) * (by1 - by0)
    union = area_a + area_b - inter_area
    return inter_area / union if union > 0 else 0.0
```

ฟังก์ชันหาพื้นที่สี่เหลี่ยมที่ซ้อนทับกันด้วยการหา max ของมุมซ้ายบนและ min ของมุมขวาล่าง แล้วคำนวณ
พื้นที่ซ้อนทับ ÷ พื้นที่รวม สคริปต์ประเมินจะเทียบกรอบที่โมเดลทำนายกับกรอบจริงของแต่ละรูปทดสอบ ถ้า
IoU ≥ 0.5 นับเป็น "ครอปสำเร็จ"

ความสำคัญ: เป็นตัวชี้วัดที่วัดผลปลายทางของระบบจริง (Crop Success Rate = 49/50 = 98%)
ซึ่งตรงกับสิ่งที่ผู้ใช้ต้องการมากกว่าคะแนน mAP ล้วน ๆ และคืนค่า 0 เมื่อกรอบไม่ซ้อนกันเลย
(กันหารด้วยศูนย์ด้วยเงื่อนไขพื้นที่รวม)

4.10 แปลงกรอบที่ผู้ใช้วาดบนเว็บเป็น YOLO format (feedback_utils.py)

ไฟล์ src/app/feedback_utils.py (เริ่มบรรทัดที่ 33)

```
def canvas_rect_to_yolo(rect: dict, canvas_w: int, canvas_h: int, img_w: int, img_h:
    int):
    """
    แปลงกรอบสี่เหลี่ยมที่วาดบน st_canvas (fabric.js object) ให้เป็น YOLO format
    (cx, cy, w, h) normalized เทียบขนาดรูปต้นฉบับจริง (img_w, img_h)


    fabric.js เก็บตำแหน่ง/ขนาดกรอบเป็น left, top, width, height ของตอนวาดครั้งแรก
    แล้วถ้าผู้ใช้ลาก–ปรับขนาดภายหลัง ขนาดจริงจะคูณด้วย scaleX/scaleY เพิ่มเข้ามา
    ต้องคูณกลับก่อนถึงจะได้ขนาดกรอบที่แสดงผลจริงบน canvas
    """
    left = rect["left"]
    top = rect["top"]
    width = rect["width"] * rect.get("scaleX", 1)
    height = rect["height"] * rect.get("scaleY", 1)


    # canvas อาจแสดงรูปย่อ/ขยายจากขนาดจริง ต้องคูณ scale กลับเป็นพิกัดรูปต้นฉบับก่อน
    scale_x = img_w / canvas_w
    scale_y = img_h / canvas_h


    x0 = left * scale_x
    y0 = top * scale_y
    x1 = (left + width) * scale_x
    y1 = (top + height) * scale_y


    # กันกรอบเกินขอบรูป (เผื่อผู้ใช้ลากทะลุขอบ canvas)
    x0, x1 = sorted((max(0, min(x0, img_w)), max(0, min(x1, img_w))))
    y0, y1 = sorted((max(0, min(y0, img_h)), max(0, min(y1, img_h))))


    cx = (x0 + x1) / 2 / img_w
    cy = (y0 + y1) / 2 / img_h
    w = (x1 - x0) / img_w
    h = (y1 - y0) / img_h
    return cx, cy, w, h
```

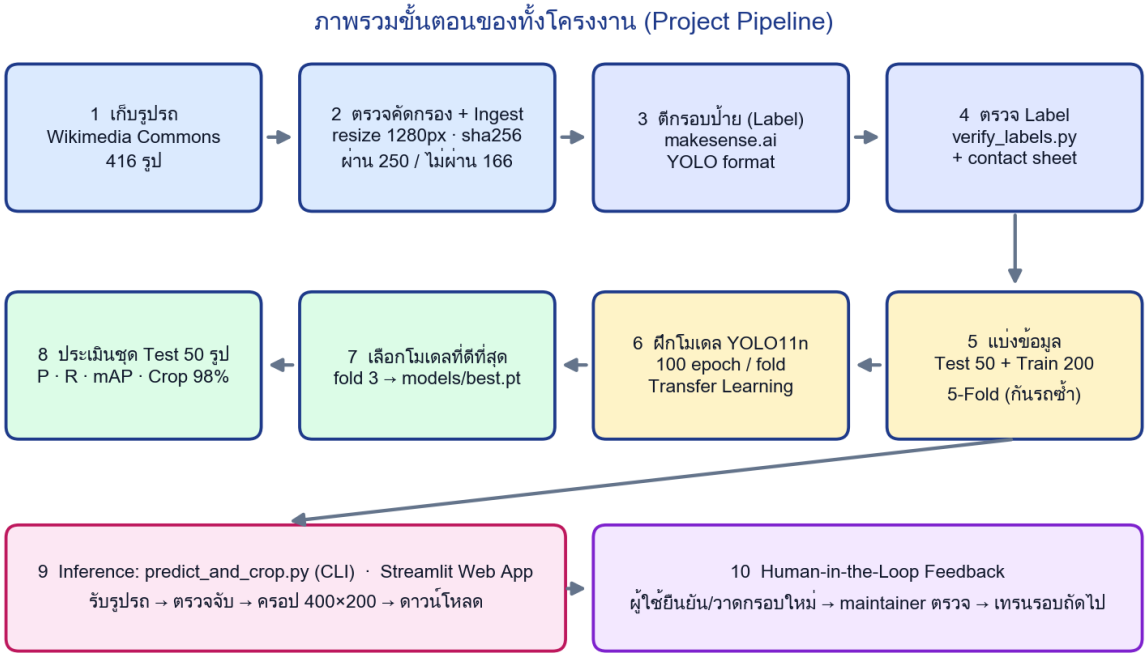
เมื่อผู้ใช้วาดกรอบป้ายเองบน canvas ของเว็บ ฟังก์ชันนี้แปลงกรอบ (ซึ่งอยู่ในพิกัดของ canvas ที่อาจถูกย่อ/ขยาย และอาจถูกลากปรับขนาดด้วย scaleX/scaleY) ให้เป็นพิกัดบนรูปต้นฉบับ กันกรอบทะลุขอบรูป แล้วจึงแปลงเป็นค่า cx, cy, w, h แบบ normalized ตามรูปแบบ YOLO

ความสำคัญ: ทำให้กรอบที่ผู้ใช้วาดบนเว็บกลายเป็นไฟล์ label ที่พร้อมใช้ฝึกโมเดลรอบถัดไปได้ทันที โดยไม่ต้องกลับไปตีกรอบใน makesense.ai อีกครั้ง (หลังผ่านการตรวจของ maintainer)

บทที่ 5 แผนภาพ (Pipeline) ของทั้งโครงงาน

5.1 ภาพรวมของทั้งโครงงาน

แผนภาพด้านล่างแสดงขั้นตอนทั้งหมดของโครงงานตั้งแต่เก็บข้อมูลจนถึงนำโมเดลไปใช้งานจริง เริ่มจากเก็บรูปและคัดกรอง ⟶ ตีกรอบ ⟶ แบ่งข้อมูล ⟶ ฝึกโมเดล ⟶ เลือกโมเดลที่ดีที่สุด ⟶ ประเมินกับชุดทดสอบ ⟶ นำไปใช้งานผ่าน Command Line และเว็บแอป โดยมีวงจร feedback จากผู้ใช้วนกลับไปพัฒนาโมเดลรอบถัดไป



รูปที่ 11 Project Pipeline: ภาพรวมขั้นตอนของทั้งโครงงาน

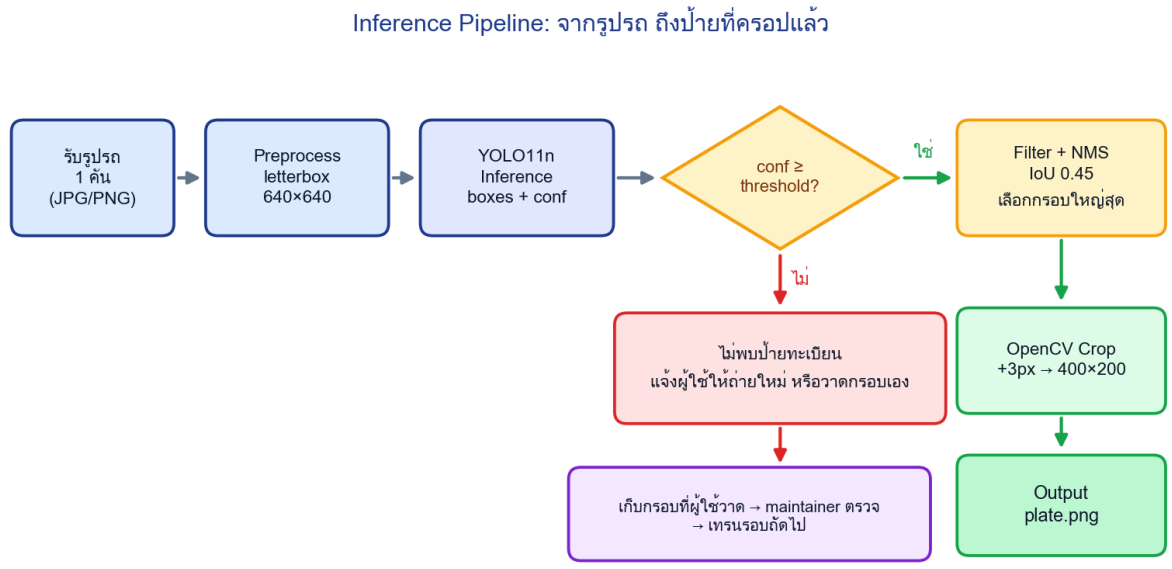
ตารางที่ 15 สรุปแต่ละขั้นตอนของโครงงานและไฟล์ที่เกี่ยวข้อง

ขั้นที่	ขั้นตอน	ไฟล์ / เครื่องมือ	ผลลัพธ์
1	เก็บรูปรถจาก Wikimedia Commons	ดาวน์โหลดตามหมวดหมู่	416 รูป
2	ตรวจคัดกรอง + นำเข้า (Ingest)	src/data_prep/ingest_images.py	250 passed / 166 rejected + manifest
3	ตีกรอบป้ายทะเบียน	makesense.ai	ไฟล์ label 250 ไฟล์ (YOLO format)

ขั้นที่	ขั้นตอน	ไฟล์ / เครื่องมือ	ผลลัพธ์
4	ตรวจความถูกต้องของ label	verify_labels.py, visualize_labels.py	ผ่าน 250/250
5	แบ่งข้อมูล Test + 5-Fold	split_folds.py	data/test, data/cv/fold_1..5
6	ฝึกโมเดล YOLO11n	src/train/train_yolo11.py	ผลต่อ fold + cv_summary.csv
7	เลือกโมเดลที่ดีที่สุด	(ในสคริปต์ฝึก)	models/best.pt (fold 3)
8	ประเมินกับชุดทดสอบ 50 รูป	evaluate_test_set.py	P, R, mAP, Crop Success 98%
9	ตรวจจับ + ครอปรูปใหม่	predict_and_crop.py, streamlit_app.py	รูปป้าย 400×200
10	รับ feedback จากผู้ใช้	feedback_utils.py, import_feedback.py	ข้อมูลใหม่สำหรับรอบถัดไป

5.2 Inference Pipeline: จากรูปรถถึงป้ายที่ครอปแล้ว

เมื่อผู้ใช้ส่งรูปเข้ามา ระบบทำงานตามลำดับดังรูป และมีจุดตัดสินใจสำคัญคือ "เจอป้ายหรือไม่" ซึ่งแยกเป็นสองเส้นทาง



รูปที่ 12 Inference Pipeline ของระบบ (เส้นทางเจอป้ายและไม่เจอป้าย)

1. **รับรูป:** ผู้ใช้ส่งรูปรถ 1 คัน (JPG/PNG) ทาง Command Line หรืออัปโหลดบนเว็บ
2. **Preprocess:** ย่อรูปแบบ letterbox เป็น 640×640 (ทำอัตโนมัติโดย Ultralytics)
3. **YOLO11n Inference:** โมเดล models/best.pt ทำนายกรอบทั้งหมดพร้อมค่าความมั่นใจ
4. **ตัดสินใจ:** ถ้าไม่มีกรอบใดมั่นใจ ≥ threshold (ค่าเริ่มต้น 0.6) ⟶ แจ้ง "ไม่พบป้ายทะเบียน"
5. **Filter + NMS:** ตัดกรอบซ้อนด้วย IoU 0.45 และเลือกกรอบที่ใหญ่ที่สุด
6. **OpenCV Crop:** ขยายขอบ 3 พิกเซล แล้วปรับเป็น 400×200
7. **Output:** ได้รูปป้ายทะเบียนพร้อมดาวน์โหลดและใช้งานต่อ



รูปที่ 13 ตัวอย่างจริง: รูปต้นฉบับพร้อมกรอบที่ตรวจจับได้ และรูปป้ายที่ครอปเป็น 400×200 พิกเซล

5.3 วงจร Human-in-the-Loop

สำหรับกรณีที่โมเดลพลาด (ไม่เจอป้าย หรือกรอบผิดตำแหน่ง)
ระบบเปิดทางให้ผู้ใช้ช่วยแก้ไขด้วยการยืนยันผลหรือวาดกรอบป้ายเอง ข้อมูลที่ผู้ใช้ส่งจะถูกเก็บแยกไว้ที่ outputs/feedback/ และต้องผ่านการตรวจด้วยตาของ maintainer ผ่านสคริปต์ import_feedback.py ก่อนนำเข้า data/raw/ เพื่อใช้เทรนรอบถัดไป **ไม่มีเส้นทางใดที่อัปเดตโมเดลที่ใช้งานอยู่โดยอัตโนมัติ** เพื่อป้องกันข้อมูลผิดพลาดทำให้โมเดลแย่ลง



รูปที่ 14 วงจร Human-in-the-Loop Feedback พร้อม Guardrail

5.4 โครงสร้างโฟลเดอร์ของโปรเจกต์

โครงสร้างโฟลเดอร์ (ตัดย่อ)

```
thai-license-plate-yolo11/
├── data/
│   ├── raw/           250 รูปที่ผ่านเกณฑ์ + dataset_manifest.csv + CREDITS.md
│   │   └── rejected/ 166 รูปที่ไม่ผ่านเกณฑ์
│   ├── labels/        ไฟล์ label (YOLO format) 250 ไฟล์
│   ├── test/          ชุดทดสอบ 50 รูป (symlink)
│   └── cv/fold_1..5/ ชุดข้อมูลสำหรับ 5-Fold CV
├── src/
│   ├── data_prep/     ingest_images, verify_labels, visualize_labels, split_folds
│   ├── train/         train_yolo11, evaluate_test_set, tune_threshold
│   ├── inference/     predict_and_crop  (Command Line)
│   ├── crop/          crop_utils        (ครอปด้วย OpenCV)
│   └── app/           streamlit_app, feedback_utils  (เว็บแอป)
├── models/best.pt     โมเดลที่ดีที่สุดจาก 5-Fold CV
├── outputs/           crops/, predictions/, feedback/, label_review/
└── docs/              รายงาน, error analysis
```

บทที่ 6 บรรยายงาน: ขั้นตอนการดำเนินงานทั้งหมด

บทนี้อธิบายการทำงานของโครงงานตามลำดับวงจรชีวิตของข้อมูลและโมเดล (Machine Learning Lifecycle) ได้แก่ Data Preparation ⟶ Data Cleaning ⟶ Data Transformation ⟶ Train Model ⟶ Trained Model ⟶ Save Model ⟶ Evaluation ⟶ Deployment และการพัฒนาต่อยอด

6.1 Data Preparation (การเตรียมข้อมูล)

ขั้นตอนนี้คือการ "หาและรวบรวมข้อมูลดิบ" ให้พร้อมสำหรับการทำความสะอาดและแปลงต่อไป

1. **กำหนดนิยามข้อมูลที่ต้องการ:** รูปรถ 1 คัน หน้าตรงหรือหลังตรง เห็นป้ายทะเบียนไทยชัดเจน
2. **ค้นหาและดาวน์โหลดจาก Wikimedia Commons** โดยเริ่มจากหมวดหมู่ที่ตรงเป้าหมายที่สุดก่อน แล้วขยายไปหมวดอื่นตามผลที่ได้ (5 รอบ รวม ~405 รูปไม่ซ้ำ)
3. **นำเข้าผ่านสคริปต์ ingest_images.py** ซึ่งทำสามอย่างอัตโนมัติ คือ กันรูปซ้ำด้วย SHA-256, ย่อรูปให้ด้านยาวไม่เกิน 1280 พิกเซล บันทึกเป็น JPEG คุณภาพ 85, และบันทึกที่มา ผู้ถ่าย สัญญาอนุญาต ลงใน dataset_manifest.csv โดยตั้งสถานะเริ่มต้นเป็น pending_review

6.2 Data Cleaning (การทำความสะอาดข้อมูล)

ข้อมูลจากอินเทอร์เน็ตมี "สิ่งปนเปื้อน" มาก จึงต้องคัดกรองอย่างเข้มงวด
ขั้นตอนนี้ใช้เวลามากที่สุดของงานเตรียมข้อมูล

ตารางที่ 16 ปัญหาของข้อมูลดิบและวิธีทำความสะอาด

ปัญหาที่พบ	วิธีจัดการ	ผลลัพธ์
รูปซ้ำ (ไฟล์เดียวกันจากหลายหมวด)	เทียบค่า SHA-256 กับ manifest	ไม่มีรูปซ้ำใน dataset
ไม่มีป้ายในรูป (รถโชว์รูม/งานแสดงรถ)	ตรวจด้วยตาทีละรูป ตั้งสถานะ rejected	reject 166 รูป ย้ายไป rejected/
ป้ายไม่ใช่มาตรฐานไทย / ถูกบัง / เล็กเกินไป	ใช้เกณฑ์เห็นป้ายครบทั้งแผ่น	เหลือ 250 รูปที่เห็นป้ายชัด
ขนาดและฟอร์แมตไม่สม่ำเสมอ	ย่อเป็น ≤1280px และบันทึก JPEG	ไฟล์มาตรฐานเดียวกัน
รถคันเดียวกันหลายรูป (เสี่ยง data leakage)	จัดกลุ่มจากชื่อไฟล์ต้นฉบับก่อนแบ่งชุดข้อมูล	ไม่มีรถซ้ำข้ามชุด/ข้าม fold

ปัญหาที่พบ	วิธีจัดการ	ผลลัพธ์
ไฟล์ label ผิดพลาด (คลิกพลาด / format ผิด)	verify_labels.py ตรวจอัตโนมัติ + contact sheet ตรวจด้วยตา	ผ่าน 250/250 รูป

สคริปต์ **verify_labels.py** ตรวจไฟล์ label ทั้งหมดอัตโนมัติ ได้แก่ มีไฟล์ label ครบทุกรูปหรือไม่, ไฟล์ว่างเปล่าหรือไม่, แต่ละบรรทัดมี 5 ค่าถูกต้องหรือไม่, หมายเลขคลาสอยู่ในช่วงหรือไม่, ค่าพิกัดอยู่ระหว่าง 0–1 หรือไม่ และขนาดกรอบเล็กหรือใหญ่ผิดปกติ (น้อยกว่า 0.05% หรือมากกว่า 50% ของพื้นที่รูป) หรือไม่ หลังจากนั้นสร้าง contact sheet (ภาพรวมของหลายรูปพร้อมกรอบ) เพื่อตรวจด้วยตาอีกชั้น

6.3 Data Transformation (การแปลงข้อมูล)

คือการแปลงข้อมูลที่สะอาดแล้วให้อยู่ในรูปแบบที่โมเดลต้องการ

ตารางที่ 17 การแปลงข้อมูลในแต่ละขั้น

การแปลง	จาก ⟶ เป็น	ทำที่ไหน
ปรับขนาดรูป	ขนาดต่าง ๆ ⟶ ด้านยาว ≤ 1280 px (JPEG q85)	ingest_images.py (ตอนนำเข้า)
แปลง label	กรอบจากเว็บ ⟶ YOLO format (cx, cy, w, h แบบ normalized 0–1)	makesense.ai (ส่งออก)
แบ่งชุดข้อมูล	250 รูป ⟶ Test 50 + 5-Fold (train 160 / val 40 ต่อ fold)	split_folds.py
สร้างโครงสร้างสำหรับ YOLO	สร้าง symlink + data.yaml ต่อ fold (nc=1, names=['license_plate'])	split_folds.py
Letterbox	รูปใดก็ได้ ⟶ 640×640 คงสัดส่วน เติมขอบเทา	Ultralytics (ตอนฝึก/ตรวจจับ)
Data Augmentation	รูปเดียวกัน ⟶ หลายรูปแบบ (mosaic, พลิก, ปรับสี, ย่อขยาย)	Ultralytics (ตอนฝึกเท่านั้น)
Normalize ค่าพิกเซล	0–255 ⟶ 0–1	Ultralytics (อัตโนมัติ)

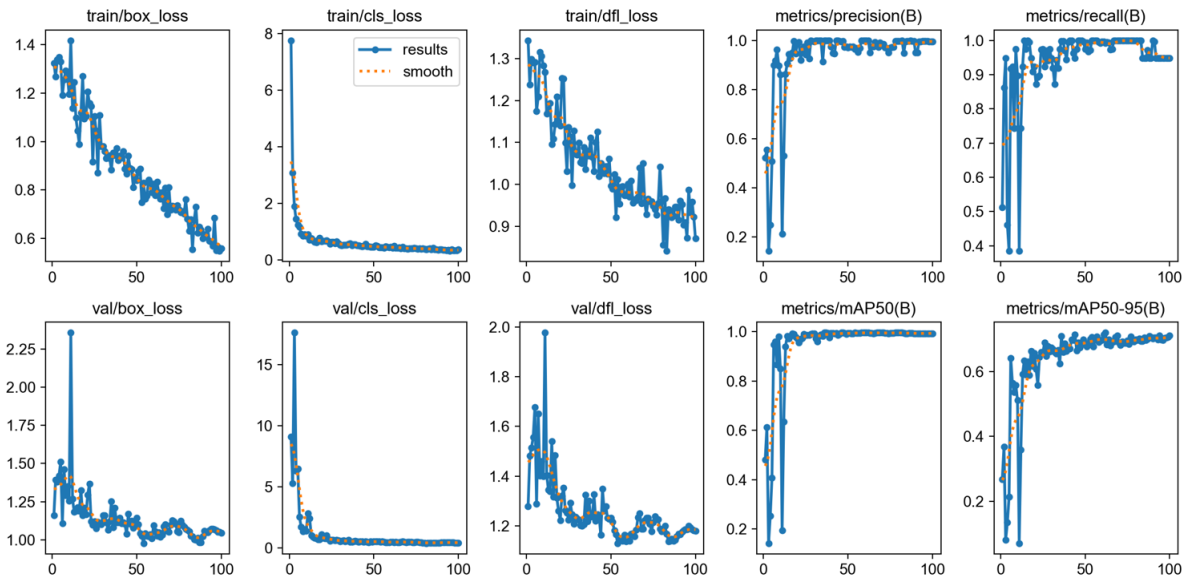
6.4 Train Model (การฝึกโมเดล)

การฝึกใช้สคริปต์ train_yolo11.py รันแบบ 5-Fold Cross-Validation คือฝึกโมเดล 5 ตัว (ตัวละ 1 fold) แต่ละตัวเริ่มจากน้ำหนัก yolo11n.pt เดียวกัน ฝึก 100 epoch โดยใช้ข้อมูลฝึก 160 รูป และตรวจกับ 40 รูปของ fold นั้น

ตารางที่ 18 พารามิเตอร์การฝึก (จาก args.yaml ของ fold 3)

พารามิเตอร์	ค่า	ความหมาย
model	yolo11n.pt	น้ำหนักตั้งต้น (Transfer Learning จาก COCO)
epochs	100	จำนวนรอบที่โมเดลเห็นข้อมูลฝึกทั้งชุด
imgsz	640	ขนาดภาพ input
batch	16	จำนวนรูปต่อการอัปเดตน้ำหนัก 1 ครั้ง
optimizer	auto	Ultralytics เลือกให้ตามจำนวนรอบ/ข้อมูล
device	mps	ใช้ GPU ของ Apple (Metal Performance Shaders)
amp	true	Mixed Precision เพื่อความเร็วและประหยัดหน่วยความจำ
patience	100	จำนวน epoch รอก่อนหยุดก่อนกำหนด (ไม่ได้หยุดก่อน)
close_mosaic	10	ปิด mosaic ใน 10 epoch สุดท้าย
seed	0	กำหนดเมล็ดสุ่มให้ผลซ้ำได้

ระหว่างฝึกในแต่ละ epoch โมเดลคำนวณ box_loss, cls_loss, dfl_loss บนชุดสอน และวัด Precision, Recall, mAP บนชุดตรวจ ผลจะถูกบันทึกลง results.csv และสร้างกราฟอัตโนมัติ
ดังรูปด้านล่างซึ่งเป็นกราฟของ fold 3 (ใช้เวลาฝึกประมาณ 14 นาทีต่อ fold บนเครื่องของผู้จัดทำ)



รูปที่ 15 กราฟผลการฝึก fold 3: loss ลดลงต่อเนื่อง ขณะที่ Precision / Recall / mAP เพิ่มขึ้นและนิ่ง

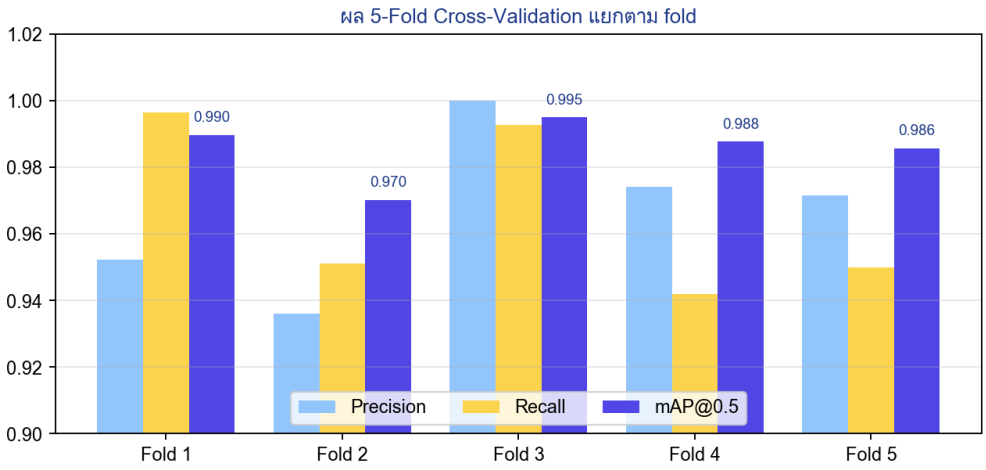
การอ่านกราฟ: เส้น train/box_loss, cls_loss, dfl_loss ลดลงเรื่อย ๆ แสดงว่าโมเดลเรียนรู้จากข้อมูลฝึกได้ ส่วน val loss ลดลงและนิ่ง ไม่พุ่งกลับขึ้น แสดงว่าไม่มีสัญญาณ overfit รุนแรง และเส้น metrics (precision, recall, mAP50) เข้าใกล้เพดานภายในช่วงกลางของการฝึก

6.5 Trained Model (ผลของโมเดลที่ฝึกแล้ว)

หลังฝึกครบ 5 fold ได้โมเดล 5 ตัว ผลของแต่ละตัวบนชุดตรวจของ fold ตัวเองเป็นดังตาราง

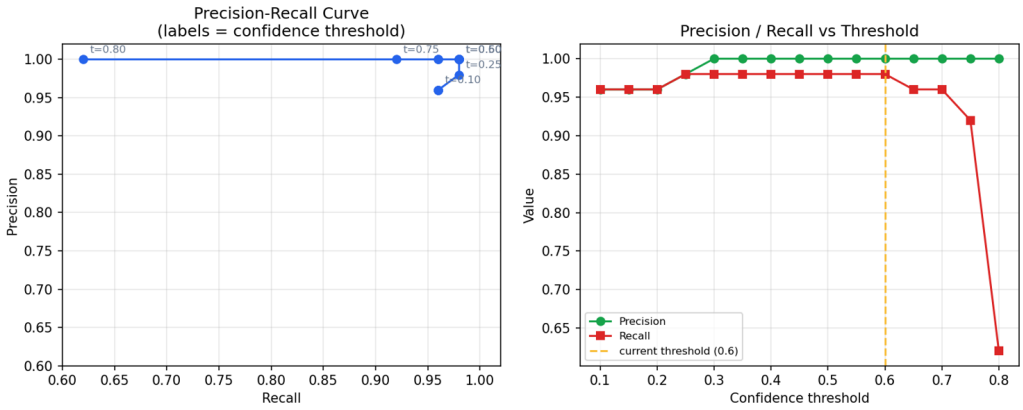
ตารางที่ 19 ผล 5-Fold Cross-Validation

Fold	Precision	Recall	mAP@0.5	หมายเหตุ
1	0.952	0.996	0.990	
2	0.936	0.951	0.970	ต่ำสุด
3	1.000	0.993	0.995	ดีที่สุด ⟶ ใช้เป็น best.pt
4	0.974	0.942	0.988	
5	0.971	0.950	0.986	
เฉลี่ย	**0.967**	**0.967**	**0.986**	



รูปที่ 16 เปรียบเทียบผลของแต่ละ fold (Precision, Recall, mAP@0.5)

การตีความ: ค่าเฉลี่ย mAP@0.5 = 0.986 และผลของแต่ละ fold อยู่ในช่วง 0.970–0.995 ความต่างระหว่าง fold น้อย แสดงว่าโมเดลเสถียร ไม่ขึ้นกับว่าข้อมูลชุดใดถูกใช้ตรวจ fold 2 ต่ำสุดเพราะบังเอิญได้รูปที่ยากกว่า (เช่นป้ายแดงหรือมุมเฉียง) อยู่ในชุดตรวจ



รูปที่ 17 กราฟ Precision–Recall ของโมเดลที่เลือกใช้

6.6 Save Model (การบันทึกและเลือกโมเดล)

ระหว่างฝึก Ultralytics บันทึกไฟล์น้ำหนักสองแบบต่อ fold คือ **last.pt** (ท้ายสุด) และ **best.pt** (epoch ที่ค่า fitness ดีที่สุดบนชุดตรวจ) เมื่อฝึกครบทุก fold สคริปต์จะเลือก fold ที่ mAP@0.5 สูงสุด (fold 3 = 0.995) แล้วคัดลอก best.pt ของ fold นั้นไปเป็น **models/best.pt** ซึ่งเป็นไฟล์เดียวที่ระบบ Inference และเว็บแอปโหลดไปใช้ (ขนาดประมาณ 5.2 MB)

> **ข้อควรระวัง**
>
> การเลือก fold ที่ดีที่สุดโดยดูจากชุดตรวจของ fold เองอาจ "ได้เปรียบ" เล็กน้อย เพราะ fold นั้นถูกเลือกจากผลบนข้อมูลชุดเดียวกัน ผู้จัดทำจึงประเมินโมเดลที่เลือกซ้ำกับ **ชุดทดสอบ 50 รูปที่ไม่เคยถูกใช้ในทุก fold** เพื่อให้ได้ผลที่เชื่อถือได้จริง (หัวข้อ 6.7)

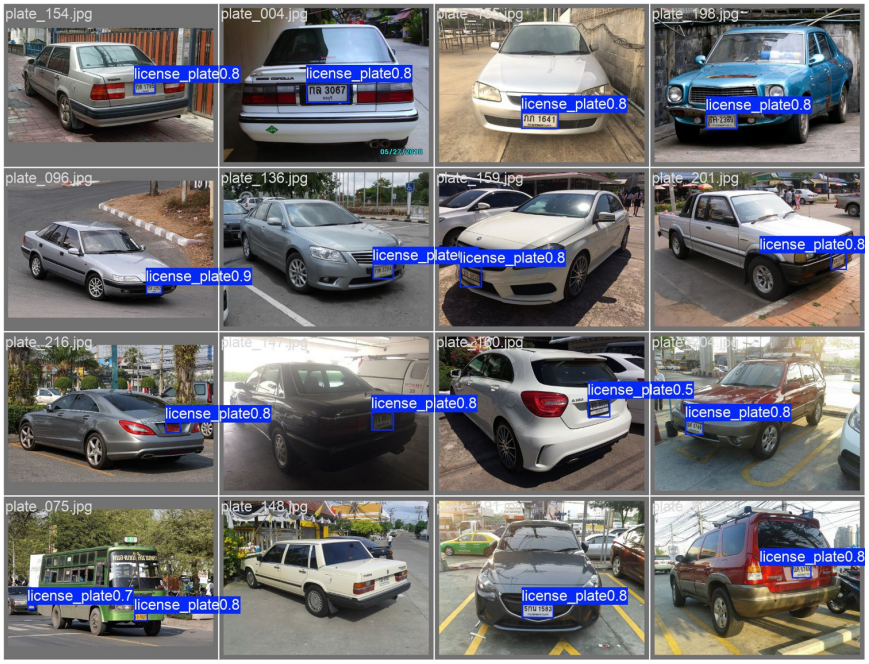
6.7 Evaluation (ประเมินผลกับชุดทดสอบ)

ชุดทดสอบ 50 รูปถูกแยกตั้งแต่ต้น ไม่ถูกใช้ทั้งการฝึกและการเลือกโมเดล จึงเป็นตัวแทนของ "รูปรถคันใหม่ที่โมเดลไม่เคยเห็น" สคริปต์ evaluate_test_set.py วัดทั้งตัวชี้วัดมาตรฐานของ YOLO และตัวชี้วัดปลายทางคืออัตราครอปสำเร็จ

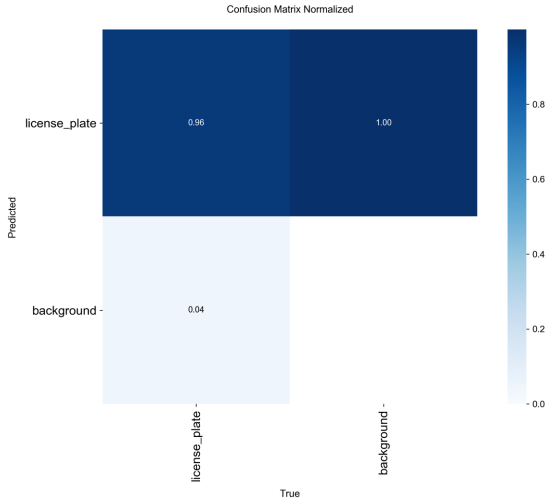
ตารางที่ 20 ผลประเมินกับชุดทดสอบ 50 รูป เทียบกับ 5-Fold CV

ตัวชี้วัด	5-Fold CV (เฉลี่ย)	ชุดทดสอบ 50 รูป
Precision	0.967	0.988
Recall	0.967	0.940
mAP@0.5	0.986	0.954
อัตราครอปสำเร็จ (IoU ≥ 0.5)	—	**98.0% (49/50)**

ผลจาก CV และชุดทดสอบใกล้เคียงกัน (ต่างกันประมาณ 3 จุดของ mAP)
ซึ่งยืนยันว่าโมเดลเรียนรู้ลักษณะทั่วไปของป้ายทะเบียนจริง ไม่ได้จำรูปในชุดฝึก (ไม่ overfit)
และสามารถใช้กับรูปรถคันใหม่ได้



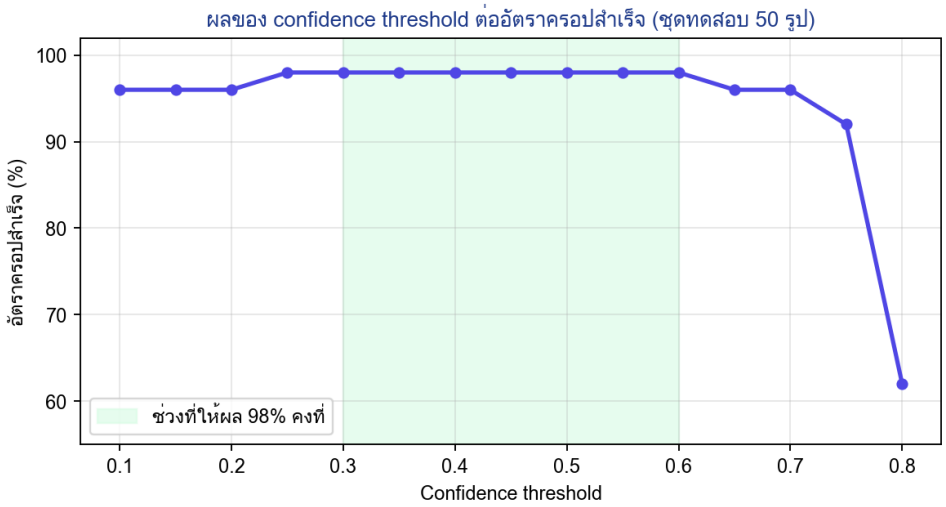
รูปที่ 18 ตัวอย่างผลการตรวจจับบนชุดทดสอบ (กรอบที่โมเดลทำนาย พร้อมค่าความมั่นใจ)



รูปที่ 19 Confusion Matrix (normalized) บนชุดทดสอบ

การเลือกค่า Confidence Threshold

ผู้จัดทำทดลองเปลี่ยนค่า confidence threshold ตั้งแต่ 0.10 ถึง 0.80 (สคริปต์ tune_threshold.py) แล้ววัดอัตราครอปสำเร็จ พบว่าช่วง **0.30–0.60 ให้ผลคงที่ที่ 98%** หากต่ำกว่านี้ (≤0.20) จะมีกรอบผิดตำแหน่งเพิ่ม (96%) และหากสูงเกินไป (≥0.75) ป้ายที่ความมั่นใจต่ำจะถูกตัดทิ้งจนอัตราตกลงชัดเจน (92% ที่ 0.75 และ 62% ที่ 0.80) จึงเลือกใช้ค่า 0.6 เป็นค่าเริ่มต้นของระบบ ซึ่งอยู่ในช่วงเสถียรและเข้มงวดพอที่จะลดกรอบมั่ว



รูปที่ 20 ผลของ confidence threshold ต่ออัตราครอปสำเร็จ

Error Analysis: กรณีที่พลาด

จากชุดทดสอบ 50 รูป มีเพียง 1 รูปที่ครอปไม่สำเร็จ คือ **plate_110.jpg** ซึ่งโมเดลไม่พบป้ายเลย วิเคราะห์สาเหตุที่เป็นไปได้ได้ 3 ข้อ

1. **ป้ายสีแดง** (ป้ายทะเบียนชั่วคราว/เต็นท์รถ) มีตัวอย่างในชุดฝึกน้อยมากเมื่อเทียบกับป้ายขาวมาตรฐาน

2. **มุมถ่ายเฉียง 3/4** ไม่ใช่มุมหน้าตรง/หลังตรงที่โครงงานกำหนด รูปทรงสี่เหลี่ยมของป้ายจึงบิดเบี้ยว
3. **แสงพลบค่ำ** ทำให้ความต่างระหว่างป้ายสีแดงกับพื้นหลังต่ำ



รูปที่ 21 รูป plate_110.jpg ที่โมเดลตรวจไม่พบป้าย (ป้ายแดง มุมเฉียง แสงพลบค่ำ)

ข้อผิดพลาดนี้สอดคล้องกับข้อจำกัดที่ระบุไว้ของโครงงาน ไม่ใช่ความผิดปกติของโมเดลที่อธิบายไม่ได้ ซึ่งแนวทางแก้ไขคือเพิ่มตัวอย่างป้ายแดงและรูปแสงน้อยในชุดฝึก และเพิ่ม augmentation ด้านความสว่าง

6.8 Deployment: การนำโมเดลไปใช้งาน

โครงงานเปิดให้ใช้งานได้ 2 ช่องทาง ใช้โมเดลไฟล์เดียวกันและตรรกะเดียวกัน (ฟังก์ชัน predict_and_crop)

ช่องทางที่ 1: Command Line

รันสคริปต์ predict_and_crop.py ได้ 2 แบบ คือ **(1) โหมดโต้ตอบ** ที่โปรแกรมถามให้พิมพ์ path ของรูปรถเอง และ **(2) ระบุรูปมากับคำสั่ง** (รูปเดียว หลายรูป หรือทั้งโฟลเดอร์) ระบบจะโหลดโมเดลครั้งเดียว ตรวจจับ ครอป และบันทึกผลลัพธ์ให้อัตโนมัติ และเลือกอุปกรณ์ประมวลผลให้เองตามลำดับ CUDA ⟶ MPS (Mac) ⟶ CPU จึงรันได้ทุกเครื่องแม้ไม่มี GPU

แบบที่ 1: ระบุรูปมากับคำสั่ง (ผลลัพธ์ที่ได้จริง)

```
$ python src/inference/predict_and_crop.py --image samples/plate_004.jpg


โหลดโมเดล: best.pt | อุปกรณ์ประมวลผล: mps
รูป: samples/plate_004.jpg
พบป้ายทะเบียน (confidence=0.82, จำนวนกรอบที่เจอ=1)
ใช้เวลาตรวจจับ: 1364 ms
บันทึกผลลัพธ์ที่: outputs/crops/plate_004_plate.jpg
สรุป: ตรวจรูปทั้งหมด 1 รูป พบป้ายและครอปสำเร็จ 1 รูป
```

แบบที่ 2: โหมดโต้ตอบ — ผู้ใช้พิมพ์ path รูปเอง (พิมพ์ q เพื่อออก)

```
$ python src/inference/predict_and_crop.py


โหลดโมเดล: best.pt | อุปกรณ์ประมวลผล: mps
โหมดโต้ตอบ — พิมพ์ path ของรูปรถ (หรือโฟลเดอร์) แล้วกด Enter | พิมพ์ q เพื่อออก


ใส่ path รูป: samples/plate_011.jpg
พบป้ายทะเบียน (confidence=0.81, จำนวนกรอบที่เจอ=1)
บันทึกผลลัพธ์ที่: outputs/crops/plate_011_plate.jpg


ใส่ path รูป: q
จบการทำงาน
```

ตารางที่ 21 ตัวเลือกของคำสั่ง predict_and_crop.py

ตัวเลือก	ความหมาย	ค่าเริ่มต้น
--image	path ของรูปรถ (ใส่ได้หลายรูป หรือเป็นโฟลเดอร์) ถ้าไม่ใส่จะเข้าโหมดโต้ตอบ	— (โหมดโต้ตอบ)
--weights	ไฟล์โมเดล	models/best.pt
--conf	ความมั่นใจขั้นต่ำ (ทิ้งกรอบที่ต่ำกว่า)	0.6
--iou	เกณฑ์ IoU ของ NMS	0.45
--device	อุปกรณ์ประมวลผล: auto / cpu / mps / 0 (CUDA)	auto (CUDA ⟶ MPS ⟶ CPU)
--out	path ไฟล์ผลลัพธ์ (ใช้กับรูปเดียว)	outputs/crops/<ชื่อไฟล์>_plate.jpg
--out-dir	โฟลเดอร์เก็บผลลัพธ์ (ใช้กับหลายรูป)	outputs/crops/

วิธีรันสำหรับผู้ตรวจ

ผู้ตรวจรันโปรแกรมบนเครื่องของตนเองได้ใน 3 ขั้นตอน (ต้องมี Python 3.9 ขึ้นไป และอินเทอร์เน็ตตอนติดตั้งครั้งแรก) โดยแพ็กเกจที่ส่งมีโมเดล models/best.pt และรูปตัวอย่างในโฟลเดอร์ samples/ ให้ครบ

ขั้นตอนการติดตั้งและรัน

```
# 1) เข้าโฟลเดอร์โปรเจกต์ แล้วสร้าง virtual environment
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate


# 2) ติดตั้ง library (ครั้งแรกครั้งเดียว)
pip install -r requirements-cli.txt
```

```
# 3) รันโปรแกรม แล้วพิมพ์ path รูปรถเองเมื่อโปรแกรมถาม
python src/inference/predict_and_crop.py
```

ไฟล์ requirements-cli.txt ล็อกเวอร์ชัน ultralytics 8.4.190 เท่ากับที่ใช้เทรนและทดสอบ เพื่อให้ผลลัพธ์ตรงกับที่รายงาน ผู้จัดทำทดสอบการติดตั้งแบบนี้ใน virtual environment ใหม่ที่แตกจากไฟล์ที่ส่งแล้ว ได้ผลตรวจจับเท่ากับในโปรเจกต์ต้นฉบับ (confidence 0.82, 0.81, 0.80 กับรูปตัวอย่างทั้ง 3 รูป)



รูปที่ 22 ผลลัพธ์ที่ได้จริงจากคำสั่งข้างต้น: ป้ายทะเบียนที่ครอปแล้ว (400×200 พิกเซล)

ช่องทางที่ 2: Web Application (Streamlit)

เว็บแอป (รันด้วย streamlit run src/app/streamlit_app.py) ให้ผู้ใช้อัปโหลดรูปแล้วเห็นผลทันที ประกอบด้วย 4 หน้า

ตารางที่ 22 หน้าต่าง ๆ ของเว็บแอป

หน้า	หน้าที่
Detect	อัปโหลดรูปรถ ดูรูปต้นฉบับพร้อมกรอบที่ตรวจพบ ดูรูปป้ายที่ครอปแล้ว รายละเอียดผลตรวจจับ (confidence, จำนวนกรอบ, เวลา) ดาวน์โหลดรูปป้าย และให้ feedback ว่าผลถูกหรือผิดตำแหน่ง
Settings	ปรับค่า Confidence threshold และ NMS IoU threshold โดยมีผลทันทีกับหน้า Detect
Model Info	แสดงข้อมูลโมเดลและผลการประเมิน (CV, ชุดทดสอบ, ที่มาของ Dataset)
Pipeline	อธิบายขั้นตอนการทำงานของระบบเป็นแผนภาพ

6.9 Feedback Loop และการพัฒนาโมเดลรอบถัดไป

หลังได้ผลลัพธ์ ผู้ใช้เลือก "ถูกต้อง" หรือ "ตำแหน่งผิด" ได้ ถ้ากรอบผิดหรือไม่เจอป้าย ผู้ใช้วาดกรอบป้ายเองบนรูป (ระบบแสดงตัวอย่างป้ายที่จะครอปแบบเรียลไทม์) แล้วส่งกรอบ ข้อมูลถูกบันทึกเป็นรูป + label + แถวใน feedback_log.csv โดยฟังก์ชัน save_feedback() ซึ่งต่อมา

maintainer ตรวจด้วยตาและนำเข้า dataset ผ่าน import_feedback.py เพื่อฝึกโมเดลรอบถัดไป
โดยแนวทางที่ถูกต้องคือรวมข้อมูลเก่าและใหม่แล้วทำ Cross-Validation ใหม่ทั้งหมด
และตรวจกับชุดทดสอบเดิมก่อนเปลี่ยนโมเดลที่ใช้งานจริง

6.10 ข้อจำกัดและแนวทางพัฒนาต่อ

ตารางที่ 23 ข้อจำกัดและแนวทางต่อยอด

ข้อจำกัด	แนวทางพัฒนาต่อ
รองรับเฉพาะรูปหน้าตรงหรือหลังตรง	เพิ่มรูปมุมเฉียง 3/4 และมุมข้าง แล้วฝึกใหม่
ป้ายสีแดงและป้ายสีอื่นมีตัวอย่างน้อย	เก็บตัวอย่างป้ายแดง/เหลือง/ป้ายพิเศษเพิ่ม
ข้อมูลฝึกเพียง 200 รูป	เพิ่มข้อมูลผ่านวงจร feedback และรูปแสงน้อย/พลบค่ำ + augmentation ด้านความสว่าง
ให้ผลเป็นรูปป้าย ไม่อ่านตัวอักษร	ต่อยอดด้วย OCR อ่านเลขทะเบียนและจังหวัด
ตรวจรถทีละ 1 คันต่อรูป	ขยายให้รองรับหลายคัน โดยคืนทุกป้ายที่เจอแทนการเลือกป้ายใหญ่สุด
สคริปต์ฝึกโมเดล (train_yolo11.py) ยังตั้ง device เริ่มต้นเป็น mps	ส่วนตรวจจับ (CLI และเว็บ) เลือกอุปกรณ์อัตโนมัติแล้ว (CUDA ⟶ MPS ⟶ CPU) เหลือปรับสคริปต์ฝึกให้เป็นแบบเดียวกัน

6.11 สรุป

โครงงานนี้พัฒนาระบบตรวจจับและครอปป้ายทะเบียนรถไทยที่ครบวงจร
ตั้งแต่การเก็บและคัดกรองข้อมูลจาก Wikimedia Commons 250
รูปพร้อมบันทึกที่มาและสัญญาอนุญาตทุกรูป การตีกรอบและตรวจสอบ label
การแบ่งข้อมูลที่กันรถซ้ำข้ามชุด การฝึกโมเดล YOLO11n ด้วย Transfer Learning แบบ 5-Fold Cross-Validation ไปจนถึงการประเมินกับชุดทดสอบที่แยกไว้

ผลลัพธ์ที่ได้คือ mAP@0.5 เฉลี่ย 0.986 จาก CV และ 0.954 บนชุดทดสอบ โดยมี **อัตราครอปสำเร็จ 98.0% (49/50 รูป)** ซึ่งผลทั้งสองชุดใกล้เคียงกัน แสดงว่าโมเดลไม่ overfit ระบบสามารถนำไปใช้ได้ทั้งผ่าน Command Line และเว็บแอป พร้อมกลไก Human-in-the-Loop
ให้ผู้ใช้ช่วยพัฒนาโมเดลในรอบถัดไปโดยไม่กระทบโมเดลที่ใช้งานอยู่
ข้อจำกัดหลักคือรูปมุมเฉียงและป้ายสีแดงที่มีตัวอย่างน้อย ซึ่งแก้ได้ด้วยการเพิ่มข้อมูลในอนาคต

บรรณานุกรม

1. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
2. Jocher, G., & Qiu, J. (2024). Ultralytics YOLO11. Ultralytics Documentation. https://docs.ultralytics.com/models/yolo11/
3. Ultralytics Documentation: YOLO Performance Metrics, Train, Predict and Val modes. https://docs.ultralytics.com/
4. Lin, T.-Y., Dollár, P., Girshick, R., He, K., Hariharan, B., & Belongie, S. (2017). Feature Pyramid Networks for Object Detection. CVPR.
5. Liu, S., Qi, L., Qin, H., Shi, J., & Jia, J. (2018). Path Aggregation Network for Instance Segmentation. CVPR.
6. He, K., Zhang, X., Ren, S., & Sun, J. (2015). Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition. IEEE TPAMI.
7. Zheng, Z., Wang, P., Liu, W., Li, J., Ye, R., & Ren, D. (2020). Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression. AAAI.
8. Li, X., Wang, W., Wu, L., et al. (2020). Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection. NeurIPS.
9. Feng, C., Zhong, Y., Gao, Y., Scott, M. R., & Huang, W. (2021). TOOD: Task-aligned One-stage Object Detection. ICCV.
10. Neubeck, A., & Van Gool, L. (2006). Efficient Non-Maximum Suppression. International Conference on Pattern Recognition (ICPR).
11. Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS.
12. Wikimedia Commons. Category: Automobiles with license plates of Thailand และหมวดหมู่ที่เกี่ยวข้อง. https://commons.wikimedia.org (รายละเอียดผู้ถ่ายและสัญญาอนุญาตของทุกรูปอยู่ใน data/raw/dataset_manifest.csv)
13. Skalski, P. (2019). Make Sense: Online tool for labeling photos. https://www.makesense.ai

14. OpenCV Documentation: Geometric Image Transformations (cv2.resize, interpolation flags). https://docs.opencv.org
15. Streamlit Documentation. https://docs.streamlit.io